

Green University of Bangladesh

Green_Eclipse

Md. Asif Samrat
Md. Din Islam
Kazi Rasel

November 7, 2026

Contents

1 Template **3**
1.1 Run command 3

2 Bit Technique **3**

3 STL Cheatsheet **3**
3.1 Comparators & Priority Queue + Set 3
3.2 Deque 3
3.3 STL Functions 3
3.4 Numeric & Array Tricks . . . 3

4 String Utilities **3**
4.1 Char Checks & ASCII (<cctype>) 3
4.2 String Manipulation 3
4.3 IO & Token Splitting 3

5 Geometry **4**
5.1 2D Shapes 4
5.2 3D Shapes 4
5.3 Polygon 5
5.4 Vectors 5
5.5 Point Input Template 5
5.6 Convex Hull 6

6 Sequence and Series **6**
6.1 Nth Term Formulas: 6
6.2 Sum of First n Terms 6
6.3 Special Sums: 6
6.4 Equation: 6

7 Combinatorics **6**
7.1 Permutations (Order Matters) 6
7.2 Permutation of Multisets . . . 6
7.3 Derangement (No element in original position) 6
7.4 Combinations (Order Doesn't Matter) 6
7.5 Stars and Bars Theorem . . . 7
7.6 Circular Permutations 7
7.7 From n Points 7
7.8 Maintaining a Condition . . . 7
7.9 Divisibility 7
7.10 Subarray 7
7.11 Tree 7
7.12 Pigeonhole Principle 7
7.13 nCr, nPr Calculate 7
7.14 Catalan Numbers and Balanced Sequences 7
7.15 Binomial Coefficients 8
7.16 Stirling Numbers 8

8 Probability / Expected Value **8**
8.1 Probability basics 8
8.2 Expected value 8
8.3 Variance (brief) 9
8.4 Contest tips 9

9 Fibonacci Numbers **9**
9.1 Definition: 9
9.2 Properties 9
9.3 Closed Form (Binet's Formula) 9
9.4 Sum of the First n Fibonacci Numbers 9
9.5 Log-factorial precompute . . 9

10 Calendar Day **10**
10.1 Leap Year: 10
10.2 Gregorian Calendar 10
10.3 Julian Calendar 10
10.4 Adjustment for January and February 10

11 Number Theory **10**
11.1 Modular Arithmetic 10
11.2 Matrix multiplication 10
11.3 GCD & Diophantine 11
11.4 Cayley's Formula 11
11.5 Binary Exponential 11
11.6 Extended Euclidean GCD . . 11
11.7 Basis Vector 11
11.8 Mobius + Mertens 12
11.9 Segmented Sieve 12
11.10Pollard Rho 12
11.11Miller Rabin 12
11.12FFT 12
11.13NTT 13
11.14Euler's Totient (Phi Function) 13
11.15Count GCD(a, b) = X 13
11.16Count & SOD of N 14
11.17Sum of Divisors [1...N] 14
11.18Power of a Prime in $n!$ 14

12 Graph **14**
12.1 Direction Arrays 14
12.2 Dijkstra's 14
12.3 Floyd-Warshall 14
12.4 Bellman-Ford 14
12.5 Kruskal's for MST 14
12.6 SCC (Kosaraju's Algo) 15
12.7 HopCroft Matching 15
12.8 DSU + RollbackDSU 15
12.9 Re-rooting Tree DP 15
12.10Binary Lifting for LCA 15
12.11HLD (Edge Weight) 16
12.12Centroid Decomposition . . . 16

13 Data Structure **17**
13.1 Ordered Set (PBDS) 17
13.2 Fenwick Tree 2D 17
13.3 Sparse Table 17
13.4 Mo's Algorithm 17
13.5 Segment Tree (Iterative) . . . 17
13.6 Lazy Propagation 18
13.7 Persistent Segment Tree . . . 18

14 Dynamic Programming **18**
14.1 0/1 Knapsack 18
14.2 Coin Change (# Ways) . . . 18
14.3 LIS 18
14.4 LCS + Reconstruction 18
14.5 Bounded Knapsack (Pref DP) 19
14.6 Bitmask, Submask & SOS DP 19
14.7 Digit DP 19
14.8 Divide & Conquer DP 19
14.9 Knuth's Optimization 19

15 String Algorithm **19**
15.1 KMP String Matching 19
15.2 Z-Function 19
15.3 Manacher 19
15.4 Min-Rotation 20
15.5 String Hashing 20
15.6 Dynamic String Hashing . . . 20
15.7 Trie 20
15.8 AhoCorasick 21
15.9 Suffix Array 21

16 Game Theory **22**
16.1 Basic Concepts 22
16.2 Standard Nim Game 22
16.3 Misère Nim 22
16.4 Bogus Nim 22
16.5 Staircase Nim 22
16.6 Sprague-Grundy Theorem . . 22
16.7 Pile Splitting Game 22
16.8 Grundy + XOR (Multiple Games) 22
16.9 Green Hackenbush (Graph Game) 22
16.10Beatty's Theorem and Wythoff's Game 22
16.11Other Useful Games 23
16.12Techniques Summary 23

17 Test & Debug **23**
17.1 Generate Random 23
17.2 Tree generator 23
17.3 Stress Testing Script 23
17.4 Debug 23
17.5 Python Basic syntax 23

(1) Template

```
#pragma GCC optimize("unroll-loops")
#pragma GCC optimize("unroll-all-loops")
#pragma GCC optimize("O3")
#pragma GCC optimize("Ofast")
#include<bits/stdc++.h>
#include <cassert>
using namespace std;
#ifdef LOCAL
#include "debug.h"
#else
#define dbg(x...)
#endif
#define i128 __int128
#define dbl double
#define ld long double
#define ull unsigned long long
#define MEM(a,x) memset(a,x,sizeof(a))
#define PI acosl(-1)
#define rep(i,a,n) for(int i=a;i<n;++i)
#define vi vector<int>
#define pb push_back
#define pii pair<int, int>
#define arr
// 128 bit output
ostream& operator<<(ostream &os, i128 x) {
    if (x < 0) return os << '-' << -x;
    if (x > 9) os << x / 10;
    return os << (char) (x % 10 + '0');
}
// Fast and better rnd()
mt19937 rng(chrono::steady_clock::now()
    .time_since_epoch().count());
// Fast I/O
ios_base::sync_with_stdio(false);
cin.tie(NULL);  cout.tie(NULL);
```

(1.1) Run command

```
// a single line & -O2, O = not zero
g++ "$1.cpp" -o "$1" -std=c++20 -O2 -
    fsanitize=address,undefined -DLOCAL
     -Wall -Wextra -Wshadow && timeout
    5s "./$1" < input.txt > output.txt
// run python
timeout 5s python3 "$1.py" < input.txt
    > output.txt 2> error.txt
// In windows replace out to exe
//cmd: chmod +x run
//run cmd: ./run fileName
```

(2) Bit Technique

```
// --- bitset<N> ---
bitset<N> b(v); b.set(); b.reset();
b.flip(); b.count(); b.any();
b.none(); b.all(); b.to_ullong();
// Iterate set bits O(N/64 + #1s):
for (int i = b._Find_first();
     i < N; i = b._Find_next(i)) {}
// --- Single Bit (0-indexed) ---
Get i-th:   (n >> i) & 1ULL
Set i-th:   n | (1ULL << i)
Clear i-th: n & ~(1ULL << i)
Flip i-th:  n ^ (1ULL << i)
Set val:    (n & ~(1ULL<<i))|(v<<i)
// --- LSB / MSB Tricks ---
Low 1 val:  n & -n
Low 0 val:  ~n & (n + 1)
Clr low 1:  n & (n - 1)
Set low 0:  n | (n + 1)
Clr trl 1s: n & (n + 1)
Set trl 0s: n | (n - 1)
Low 1s msk: n ^ (n - 1)
Is Pow2:    n > 0 && !(n & (n - 1))
// --- Block Ops (0-idx, incl) ---
Clr 0..i:   n & (~0ULL << (i + 1))
Keep 0..i:  n & ((1ULL<<(i+1)) - 1)
1s in l..r: ((1ULL<<(r-l+1))-1)<<l
// --- Builtins ---
__builtin_popcountll(n) // # of 1s
__builtin_clzll(n) // Lead 0s (n!=0)
__builtin_ctzll(n) // Trail 0s(n!=0)
__builtin_ffsll(n) // 1+ctz (0->0)
__builtin_parityll(n) // Odd 1s?
__lg(n) // MSB idx: 63-clzll (n!=0)
// --- Identities ---
a + b = (a ^ b) + 2 * (a & b)
a + b = (a | b) + (a & b)
a ^ b = (a | b) - (a & b)
n % 2^i   = n & ((1 << i) - 1)
1^2^3^...^(4k-1) = 0
```

(3) STL Cheatsheet

(3.1) Comparators & Priority Queue + Set

```
// Lambda Sort (Asc second, Desc first)
sort(all(v), [](auto& a, auto& b) {
    if (a.second != b.second)
        return a.second < b.second;
    return a.first > b.first;
});
// Min-Heap PQ
priority_queue<ll, vector<ll>,
    greater<ll>> pq_min;
// Custom Comparator for PQ / Set
// Replace array<int, 2> -> int/pll
auto cmp = [](const array<int, 2>& a,
    const array<int, 2>& b) {
    return a[1] > b[1]; // Min-PQ |
        Desc-Set
};
priority_queue<array<int, 2>,
    vector<array<int, 2>>, decltype(cmp
        )> pq(cmp);
set<array<int, 2>, decltype(cmp)> st(
    cmp);
```

(3.2) Deque

```
deque<int> dq;
dq.push_back(x);   dq.push_front(x);
dq.pop_back();     dq.pop_front();
dq.front();        dq.back();
dq[i]; // O(1) random access
dq.insert(dq.begin() + i, val);
dq.erase(dq.begin() + i);
dq.erase(dq.begin()+l, dq.begin()+r);
dq.clear();        dq.size();
```

(3.3) STL Functions

```
// Coordinate Compression
sort(all(v));
v.erase(unique(all(v)), v.end());
lower_bound(all(v), x)-v.begin(); // id
// Binary Search & Bounds
bool ok = binary_search(all(v), x);
auto it1 = lower_bound(all(v), x); //
    >=x
auto it2 = upper_bound(all(v), x); // >
    x
// Permutations & Rearrangement
nth_element(v.begin(), v.begin()+k, v.
    end()); // O(N)
rotate(v.begin(),v.begin()+k,v.end());
auto [mn, mx] = minmax_element(all(v));
do {
    // process v (MUST sort v first!)
} while (next_permutation(all(v)));
```

(3.4) Numeric & Array Tricks

```
iota(all(v), 0); // fills 0, 1, 2...
ll sum = accumulate(all(v), 0LL);
// Prefix Sums & Adjacent Differences
vector<ll> dp(n);
partial_sum(all(v),dp.begin());
adjacent_difference(all(v),dp.begin());
// Fast 2D fill & non-zero memset
fill(&dp[0][0], &dp[0][0] + N*M, INF);
memset(d, 0x3f, sizeof(d)); // ~1e9
memset(d, 0xc0, sizeof(d)); // ~-1e9
```

(4) String Utilities

(4.1) Char Checks & ASCII (<cctype>)

```
isalpha(c); // a-z or A-Z
isdigit(c); // 0-9
isalnum(c); // a-z, A-Z, or 0-9
islower(c); isupper(c); isspace(c);
c = tolower(c); c = toupper(c);
// Char <-> Int & Case Toggle
int d = c - '0'; char ch = d + '0';
int id = c - 'a'; // 0 to 25
c ^= 32; // toggle a <-> A
string s(n, 'a'); // "aaa...a"
```

(4.2) String Manipulation

```
transform(all(s),s.begin(), ::toupper);
transform(all(s),s.begin(), ::tolower);
// String <-> Number (stoll for 64-bit)
ll x = stoll(s);      ld d = stold(s);
ll b = stoll(s, 0, 2); // binary to dec
string s = to_string(x);
// Substring, Erase, Insert, Replace
string sub = s.substr(pos, len);
string suf = s.substr(pos); // to end
s.erase(pos, len); // O(N) index erase
s.insert(pos, "abc");
s.replace(pos, len, "new_str");
// Search & C++20 Prefix/Suffix Check
if (s.find("ab") != string::npos) {}
size_t p = s.find("ab", start_idx);
size_t r = s.rfind("ab"); // last match
bool ok1 = s.starts_with("pre");// C
    ++20
bool ok2 = s.ends_with("suf");// C++20
```

(4.3) IO & Token Splitting

```
// Read line after cin >> n
string s;
getline(cin >> ws, s);
// Split by spaces
stringstream ss(s);
string tok;
while (ss >> tok) v.push_back(tok);
// Split by custom delimiter (e.g.,
    ',')
while (getline(ss, tok, ',')) {
    v.push_back(tok);
}
```

(5) Geometry

(5.1) 2D Shapes

(5.1.1) Triangle

•Perimeter and Semiperimeter:

$$P = a + b + c, \quad s = \frac{a+b+c}{2}$$

Perimeter and Semiperimeter:

$$P = a + b + c, \quad s = \frac{a+b+c}{2}$$

To find the lengths of the sides a, b, and c of the triangle, given the coordinates of the vertices $A(x_1, y_1)$, $B(x_2, y_2)$, and $C(x_3, y_3)$, we use the distance formula between two points (x_1, y_1) and (x_2, y_2):

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

•Area using Heron's formula:

$$A = \sqrt{s(s-a)(s-b)(s-c)}$$

•Area using two sides and the included angle:

$$A = \frac{1}{2}ab\sin\theta$$

•Right triangle:

$$A = \frac{1}{2}bh, \quad c^2 = a^2 + b^2 \text{ [Pythagorean Theorem]}$$

•Equilateral triangle (side = a):

$$\text{Height} = \frac{\sqrt{3}}{2}a, \quad A = \frac{\sqrt{3}}{4}a^2$$

•Isosceles triangle (equal sides = a, base = b):

$$\text{Height} = \frac{\sqrt{4a^2 - b^2}}{2}, \quad A = \frac{b}{4}\sqrt{4a^2 - b^2}$$

•Coordinate Geometry Formulas:

$$A = \frac{1}{2}\left|x_1(y_2 - y_3) + x_2(y_3 - y_1) + x_3(y_1 - y_2)\right|$$

$$\text{Centroid} = \left(\frac{x_1 + x_2 + x_3}{3}, \frac{y_1 + y_2 + y_3}{3}\right)$$

•Circumcenter of a Triangle:

$$D = 2(x_1(y_2 - y_3) + x_2(y_3 - y_1) + x_3(y_1 - y_2))$$

$$U_x = \frac{(x_1^2 + y_1^2)(y_2 - y_3) + (x_2^2 + y_2^2)(y_3 - y_1)}{D} + \frac{(x_3^2 + y_3^2)(y_1 - y_2)}{D}$$

$$U_y = \frac{(x_1^2 + y_1^2)(x_3 - x_2) + (x_2^2 + y_2^2)(x_1 - x_3)}{D} + \frac{(x_3^2 + y_3^2)(x_2 - x_1)}{D}$$

$$\text{Circumcenter} = (U_x, U_y)$$

•Triangle Radii:

$$\text{Inradius} = \frac{A(area)}{s}, \quad \text{Circumradius} = \frac{abc}{4A}$$

•Miscellaneous:

Altitude (the perpendicular line segment from a vertex to the opposite side) $= \dfrac{2A}{\text{base}}$

Equilateral triangle side from height h :

$$a = \frac{2h}{\sqrt{3}}$$

If two sides x and angle 120° : 3rd side $= \sqrt{3}x$

(5.1.2) Quadrilateral

•General Quadrilateral:

$$\text{Area} = 2 \cdot \text{TriangleArea}$$

•Rectangle:

$$\text{Perimeter} = 2(l + w), \quad \text{Length} = \frac{P}{2} - w$$

$$\text{Width} = \frac{P}{2} - l, \quad \text{Area} = l \cdot w$$

•Parallelogram:

$$\text{Perimeter} = 2(\text{length} + \text{base}),$$
$$\text{Area} = \text{base} \cdot \text{height}$$

•Square:

$$\text{Perimeter} = 4a, \quad \text{Area} = a^2, \quad \text{Diagonal} = \sqrt{2}a$$

•Rhombus:

$$\text{Perimeter} = 4a, \quad \text{Area} = \frac{d_1 \cdot d_2}{2}$$

•Trapezium:

$$\text{Perimeter} = a + b + c + d, \quad \text{Area} = \frac{(a+b) \cdot h}{2}$$

(5.1.3) Circle

•Circle (General Equation):

$$(x - a)^2 + (y - b)^2 = r^2 \ [(a, b) \textit{ center point}]$$
$$S = x^2 + y^2 + 2gx + 2fy + c = 0$$
$$\text{Center} = (-g, -f), \quad \text{Radius} = \sqrt{g^2 + f^2 - c}$$
$$\text{X part} = 2\sqrt{g^2 - c}, \quad \text{Y part} = 2\sqrt{f^2 - c}$$

•Circle Fundamentals & Conversions:

$$\text{Circumference} = 2\pi r$$
$$\text{Area} = \pi r^2$$
$$\theta_{\text{rad}} = \theta_{\text{deg}} \times \frac{\pi}{180^\circ}$$
$$1 \text{ Radian} = \frac{180^\circ}{\pi} \text{ degrees}$$

•Arc Length (s):

$$s = \frac{\theta}{360^\circ} \cdot 2\pi r \quad [\theta \text{ in deg}]$$
$$s = r\theta \quad [\theta \text{ in rad}]$$

•Sector Area (A):

$$A = \frac{\theta}{360^\circ} \cdot \pi r^2 \quad [\theta \text{ in deg}]$$
$$A = \frac{1}{2}r^2\theta \quad [\theta \text{ in rad}]$$

•Circle & Chord Length (c):

$$c = 2R\sin\left(\frac{\theta}{2}\right) \text{ (given angle } \theta \text{ in radians)}$$
$$c = 2\sqrt{R^2 - d^2} \text{(dist. } d \text{ from center, } d \le R)$$

(5.1.4) Ellipse

•Ellipse:

$$\text{Perimeter (approx)} = \pi(a + b), \quad \text{Area} = \pi ab$$

(5.1.5) Hexagon

•Regular Hexagon (6 equal sides):

$$\text{Perimeter} = 6a,$$
$$\text{Area} = 6 \cdot \text{EquilateralTriangleArea}$$
$$\text{Area (general)} = \frac{na^2 \cot\left(\frac{180}{n}\right)}{4}$$

(5.2) 3D Shapes

•Cuboid (Rectangle box):

$$\text{Diagonal} = \sqrt{a^2 + b^2 + c^2}$$
$$\text{Total Surface Area (TSA)} = 2(ab + bc + ca)$$
$$\text{Lateral Surface Area (LSA)} = 2h(l + b)$$
$$\text{Volume} = abc$$

•Cube (Square box):

$$\text{Volume} = a^3, \quad \text{LSA} = 4a^2, \quad \text{TSA} = 6a^2$$

•Sphere (3D Circle):

$$\text{Volume} = \frac{4}{3}\pi r^3, \quad \text{LSA} = 4\pi r^2, \quad \text{TSA} = 4\pi r^2$$

•Hemisphere (half of a sphere):

$$\text{Volume} = \frac{2}{3}\pi r^3, \quad \text{LSA} = 2\pi r^2, \quad \text{TSA} = 3\pi r^2$$

•Cylinder:

$$\text{Volume} = \pi r^2 h, \quad \text{LSA} = 2\pi rh,$$
$$\text{TSA} = 2\pi r(r + h)$$

•Cone:

$$\text{Volume} = \frac{1}{3}\pi r^2 h, \quad \text{LSA} = \pi rl,$$
$$\text{TSA} = \pi r(r + l)$$

(5.2.1) Prism

•Triangular Prism Volume:

$$\text{Volume} = \text{Area of triangle} \times \text{Height}$$
$$= \left(\frac{1}{2} \times \text{base} \times \text{height}\right) \times \text{Height}$$

where base = length of triangle base, height = height of triangle, Height = height of prism.

•Surface Area of Triangular Prism:

$$\text{Surface Area} = 2 \times \text{Area} + \text{Perimeter} \times \text{Height}$$

(5.3) Polygon

•Interior and Exterior Angles (for an n-sided polygon):

$$\text{Sum of interior angles} = (n-2)\times 180^\circ$$
$$\text{Each interior angle (regular)} = \frac{(n-2)\times 180^\circ}{n}$$
$$\text{Each exterior angle (regular)} = \frac{360^\circ}{n}$$

•Perimeter:

$$\text{Perimeter of a regular polygon} = n \times s$$

•Area of a Regular Polygon:

$$A = \frac{n \times s^2}{4\tan\left(\frac{\pi}{n}\right)}$$
$$A = \frac{1}{2} \times \text{Perimeter} \times a$$

•Number of Diagonals:

$$D = \frac{n(n-3)}{2}$$

•Apothem: (The perpendicular distance from the center to a side in a regular polygon)

$$a = \frac{s}{2\tan\left(\frac{\pi}{n}\right)} \quad \text{[using side length]}$$
$$a = R\cos\left(\frac{\pi}{n}\right) \quad \text{[using radius]}$$

•Polygon Area (Shoelace Formula):
Given polygon vertices $\{p_i\}$,

$$\text{Area} = \frac{1}{2}\left|\sum_{i=0}^{n-1}(x_i y_{i+1} - x_{i+1}y_i)\right|$$

•Pick's Theorem (Lattice Points):
$A = I + \frac{B}{2} - 1 \iff 2A = 2I + B - 2$
$I = \frac{2A-B+2}{2}, \quad B = 2A - 2I + 2$
(A: Area, I: Interior, B: Boundary $= \sum \gcd(|\Delta x|, |\Delta y|)$, $j = (i+1) \bmod n$, $\Delta x = |x_i - x_j|$, $\Delta y = |y_i - y_j|$.)

•Polygon Perimeter:

$$\text{Perimeter} = \sum_{i=0}^{n-1}\sqrt{(x_{i+1}-x_i)^2 + (y_{i+1}-y_i)^2}$$

•Centroid of Simple Polygon:

$$C_x = \frac{1}{6A}\sum_{i=0}^{n-1}(x_i + x_{i+1})(x_i y_{i+1} - x_{i+1}y_i)$$
$$C_y = \frac{1}{6A}\sum_{i=0}^{n-1}(y_i + y_{i+1})(x_i y_{i+1} - x_{i+1}y_i)$$

where A is the polygon area using the Shoelace Formula.

•Convex Hull: Use algorithms like *Graham Scan* or *Monotone Chain* to find the smallest convex polygon enclosing a set of points.

(5.3.1) Lines

•Line Equations:
$y = mx + c$ (Straight line)
Slope: $m = \frac{y_2 - y_1}{x_2 - x_1}$
$m = \tan\theta$ (Slope in terms of angle)
$m_1 \times m_2 = -1$ (Perpendicular lines)
$m = \infty$ (Vertical line)
$m = 0$ (Horizontal line)

•Distance From Point to Line(ax + bx + c = 0):

$$d = \frac{|ax_1 + by_1 + c|}{\sqrt{a^2+b^2}} \quad \text{(Perpendicular distance)}$$
$$d_{\text{signed}} = \frac{ax_1 + by_1 + c}{\sqrt{a^2+b^2}} \quad \text{(Signed distance)}$$

•Distance From a Point to a Line (Two-Point Form):

$$d = \frac{|(x_2-x_1)(y_1-y_0) - (x_1-x_0)(y_2-y_1)|}{\sqrt{(x_2-x_1)^2 + (y_2-y_1)^2}}$$
$$d = \frac{|(P_2-P_1)\times(P-P_1)|}{|P_2-P_1|} \quad \text{(Vector form)}$$

•Intersection Ratio:
A point dividing segment between (x_1, y_1) and (x_2, y_2) in ratio $m_1 : m_2$:

$$x = \frac{m_1x_2 + m_2x_1}{m_1+m_2}, \quad y = \frac{m_1y_2 + m_2y_1}{m_1+m_2}$$

•Parallel and Perpendicular Lines:
–**Parallel line**: change c to another constant k in $y = mx + c$
–**Perpendicular line**: swap coefficients and change sign in line form
$ax + by + c = 0 \Rightarrow bx - ay + k = 0$

•Quadrant-based Angle θ for point $P(x, y)$:

$$\theta = \begin{cases} \tan^{-1}(\frac{y}{x}), & \text{if } x>0, y>0 \text{ (1st quad)} \\ \pi - \tan^{-1}(|\frac{y}{x}|), & \text{if } x<0, y>0 \text{ (2nd quad)} \\ \pi + \tan^{-1}(|\frac{y}{x}|), & \text{if } x<0, y<0 \text{ (3rd quad)} \\ 2\pi - \tan^{-1}(|\frac{y}{x}|), & \text{if } x>0, y<0 \text{ (4th quad)} \end{cases}$$

•Cartesian and Polar Coordinates:

$$x = r\cos\theta, y = r\sin\theta, \; r = \sqrt{x^2+y^2}, \; \theta = \tan^{-1}\left(\frac{y}{x}\right)$$

(5.4) Vectors

•Dot Product: For two vectors $\mathbf{a} = (a_x, a_y, a_z)$ and $\mathbf{b} = (b_x, b_y, b_z)$, the dot product gives a scalar representing how much $\mathbf{a}$ aligns with $\mathbf{b}$:

$$\mathbf{a}\cdot\mathbf{b} = a_xb_x + a_yb_y + a_zb_z$$

Use: Calculate angle between vectors or check orthogonality ($\mathbf{a}\cdot\mathbf{b} = 0$).

•Cross Product: The cross product produces a vector perpendicular to both $\mathbf{a}$ and $\mathbf{b}$:

$$\mathbf{a}\times\mathbf{b} = (a_yb_z - a_zb_y, \quad a_zb_x - a_xb_z, \quad a_xb_y - a_yb_x)$$

Uses: *Area of Parallelogram:* Given two vectors $\mathbf{a} = (a_x, a_y)$ and $\mathbf{b} = (b_x, b_y)$, the area of the parallelogram they form is the absolute value of the cross product:

$$\text{Area} = |\mathbf{a}\times\mathbf{b}| = |a_xb_y - a_yb_x|$$

In 3D, the area is the magnitude of the cross product vector:

$$\text{Area} = \|\mathbf{a}\times\mathbf{b}\| = \sqrt{(a_yb_z - a_zb_y)^2 + (a_zb_x - a_xb_z)^2 + (a_xb_y - a_yb_x)^2}$$

This value represents the parallelogram's area formed by $\mathbf{a}$ and $\mathbf{b}$.

Orientation:
Find the normal vector and calculate the area of the parallelogram formed by $\mathbf{a}$ and $\mathbf{b}$. To determine the orientation of three points A, B, C, compute the cross product of vectors $\overrightarrow{AB}$ and $\overrightarrow{AC}$:

$$\overrightarrow{AB} = \langle x_2 - x_1,\ y_2 - y_1,\ z_2 - z_1\rangle$$
$$\text{orientation} = (\overrightarrow{AB}) \times (\overrightarrow{AC})$$

- If orientation > 0, points are arranged counterclockwise.
- If orientation < 0, points are arranged clockwise.
- If orientation $= 0$, points are collinear.
Essential for convex hull construction, polygon validation, and line intersection detection.

•Projection: Projection of vector $\mathbf{A}$ on $\mathbf{B}$ finds the component of $\mathbf{A}$ in the direction of $\mathbf{B}$:

$$\text{scalar} = \frac{\mathbf{A}\cdot\mathbf{B}}{\mathbf{B}\cdot\mathbf{B}}, \quad \mathbf{C} = \text{scalar} \times \mathbf{B}$$

where $0 \le \text{scalar} \le 1$ if $\mathbf{C}$ lies between $\mathbf{0}$ and $\mathbf{B}$. *Use:* Find shadow or component of a vector along another vector.

(5.5) Point Input Template

```
struct Point {
    ll x, y;
    Point operator -(const Point &p)
        const {
        return Point{x - p.x, y - p.y};
    }
    void operator -=(const Point &p) {
        x -= p.x; y -= p.y;
    }
    ll operator *(const Point &p) const
        {
        return x * p.y - y * p.x;
    }
    double norm() const {
        return sqrt(x*x + y*y);
    }
    ll cross(const Point &b, const
        Point &c) const {
        return (b - *this) * (c - *this
            );
    }
    ll dot(const Point &p) const {
        return x * p.x + y * p.y;
    }
    ll dist2() const {
        return x * x + y * y;
    }
    // P1 inside, a and b or not...
    bool isInside (const Point &a,
        const Point &b) const {
        return min(a.x, b.x) <= x && x
            <= max(a.x, b.x) &&
```

```
        min(a.y, b.y) <= y && y <= max(a
            .y, b.y);
    }
    bool operator<(const Point &a)
        const {
        return make_pair(x, y) <
            make_pair(a.x, a.y);
    }
    friend istream& operator>>(istream
        &is, Point &p) {
        return is >> p.x >> p.y;
    }
    friend ostream& operator<<(ostream
        &os, const Point &p) {
        os << p.x << " " << p.y;
        return os;
    }
};
inline int getOrientation(ll x) {
    return ((x > 0) - (x < 0));
}
bool isIntersect(Point p1, Point p2,
    Point p3, Point p4) {
    // First check Co-linear..
    if ((p2 - p1) * (p4 - p3) == 0) {
        if (p1.cross(p2, p3)) {
            return 0; //No Intersect
        }
        // Check Box-Overlapping..
        for (int rep = 0; rep < 2; ++
            rep) {
            if ((max(p1.x, p2.x) < min(
                p3.x, p4.x)) || (max(p1
                .y, p2.y) < min(p3.y,
                p4.y))) {
                return 0;//No Intersect
            }
            swap(p1, p3); swap(p2, p4);
        }
        return 1;
    }
    // now check side using sign..
    for (int rep = 0; rep < 2; ++rep) {
        int sign = getOrientation(p1.
            cross(p2, p3));
        int sign2 = getOrientation(p1.
            cross(p2, p4));

        if ((sign > 0 && sign2 > 0) ||
            (sign < 0 && sign2 < 0)) {
            return 0; // No
                Intersection..
        }
        swap(p1, p3); swap(p2, p4);
    }
    return 1; // Intersection..
}
// using....
// Point p1, p2, p3, p4 or vector<Point
    > p(4)
// p3.isInside(p1, p2), p4.isInside(p1,
    p2)
// isIntersect(p1, p2, p3, p4)
// p1.cross(p2, p3), p1->p2->p3
// lld x = 1 / p1.norm();
```

(5.6) Convex Hull

```
/*
- Computes the Convex Hull of a set of
    2D points using Graham's Scan.
- Time: O(N*logN)
*/
void ConvexHull() {
    int n; cin >> n;
    vector<Point> p(n);
    for (auto &x : p) { cin >> x; }
    sort(all(p));
    vector<Point> hull;
    for (int rep = 0; rep < 2; ++rep) {
        const int m = hull.size();
        for (auto &c : p) {
            while ((int) hull.size() -
                m > 1) {
                Point a = hull.end()
                    [-2];
                Point b = hull.end()
                    [-1];
                int sign =
                    getOrientation(a.
                    cross(b, c));
                if (sign <= 0) break;
                    // good, or
                    collinear
                hull.pop_back();//
                    remove b
            }
            hull.push_back(c);
        }
        hull.pop_back(); // end point
            count twice, so remove
        reverse(all(p)); // calculate
            for down part
    }
    cout << hull.size() << "\n";
    for (auto &x : hull) cout<<x<<"\n";
}
```

(6) Sequence and Series

(6.1) Nth Term Formulas:

Arithmetic: $a_n = a + (n-1)d$

Geometric: $a_n = ar^{n-1}$

(6.2) Sum of First n Terms

Arithmetic: $S_n = \frac{n}{2}(2a + (n-1)d)$

Geometric (if $r < 1$): $S_n = \frac{a(1-r^n)}{1-r}$

Geometric (if $r > 1$): $S_n = \frac{a(r^n-1)}{r-1}$

(6.3) Special Sums:

Sum of first n natural numbers: $\frac{n(n+1)}{2}$

Sum of squares of first n natural numbers: $\frac{n(n+1)(2n+1)}{6}$

Sum of squares of first n even numbers: $\frac{2n(n+1)(2n+1)}{3}$

Sum of squares of first n odd numbers: $\frac{n(2n+1)(2n-1)}{3}$

Sum of cube of first n natural numbers: $\frac{n^2(n+1)^2}{4}$

Sum of cube of first n natural numbers: $\frac{n^2(n+1)^2}{4}$

Sum of quadric of first n natural numbers: $\frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$

(6.4) Equation:

$$ax^2+bx+c=0 \quad \Rightarrow \quad x = \frac{-b \pm \sqrt{b^2-4ac}}{2a}$$

(7) Combinatorics

(7.1) Permutations (Order Matters)

Permutation of all n elements: $P(n) = n!$

Permutation of r elements chosen from n,

no repetition: $P(n,r) = \frac{n!}{(n-r)!}$

with repetition allowed: $P(n,r) = n^r$

Use: Example: arrange 2 from A, B, C: AB, BA, AC, CA, BC, CB.

(7.2) Permutation of Multisets

When a set contains n objects where:

- P_i objects are of the i-th kind (indistinguishable)

then the total number of distinct permutations is given by:

$$\frac{n!}{P_1! \cdot P_2! \cdot \cdots \cdot P_k!}$$

(7.3) Derangement (No element in original position)

$$!n = \left\lfloor \frac{n!}{e} + 0.5 \right\rfloor$$
$$= n! \left(1 - \frac{1}{1!} + \frac{1}{2!} - \frac{1}{3!} + \cdots + \frac{(-1)^n}{n!}\right)$$

Use: Counts permutations with no fixed points. Useful in problems like secret Santa or seat rearrangements.

If a permutation has at most k deranged (misplaced) positions:

$$\text{Total} = \sum_{r=0}^{k} \binom{n}{r} \cdot !\, r$$

It counts permutations of n elements with at most k misplaced positions. $\binom{n}{r}$ chooses which positions are misplaced, and $!\, r$ counts derangements of those positions.

(7.4) Combinations (Order Doesn't Matter)

Without repetition: $C(n,r) = \frac{n!}{r!\,(n-r)!}$

With repetition (multiset selection):

$$C(n+r-1,r) = \frac{(n+r-1)!}{r!\,(n-1)!}$$

Use: Example: arrange 2 from A, B, C: AB, AC, BC.

(7.5) Stars and Bars Theorem

Number of solutions to $x_1 + x_2 + \cdots + x_k = n$,

$x_i \geq 0$: $C(n+k-1, k-1)$
$x_i > 0$: $C(n-1, k-1)$
$a > 1, b > 3$:
$C(n+k-1-(1+3), k-1)$

Use: Distribute identical items into boxes with or without emptiness allowed.

(7.6) Circular Permutations

When clockwise and anticlockwise arrangements are considered,

same: $\frac{(n-1)!}{2}$
different: $(n-1)!$

Use: Arrange people/items around a round table or circle.

(7.7) From n Points

Number of distinct lines from n points:

$$C(n,2) = \frac{n(n-1)}{2}$$

Number of triangles: $C(n,3)$
Number of rectangles (in grid points): $C(n,4)$
Number of diagonals in an n-sided polygon:

$$C(n,2) - n = \frac{n(n-3)}{2}$$

Use: Geometric and combinational problems involving points and polygons.

(7.8) Maintaining a Condition

$$\left(\frac{n!}{(n-r)!}\right) \times \left(\frac{m}{n}\right)$$

Use: This can represent weighted permutations or conditional selections where m/n is a probability or fraction related to selection.

(7.9) Divisibility

Statement. For integers n and r with $0 \leq r \leq n$,

$$\frac{n!}{(n-r)!} \quad \text{is divisible by} \quad r!\,,$$

Algebraic proof. By definition of the binomial coefficient,

$$\binom{n}{r} = \frac{n!}{r!\,(n-r)!}.$$
$$= \frac{n-r+1}{1} \times \frac{n-r+2}{2} \ldots \times \frac{n}{r}.$$

Example. Take $n = 6$, $r = 3$. Then

$$\frac{6!}{(6-3)!} = \frac{720}{6} = 120, \qquad r! = 3! = 6,$$
$$\frac{120}{6} = 20 = \binom{6}{3}, \text{ an integer.}$$

(7.10) Subarray

All possible subarray count:

$$a_1, a_2, .., a_n = \frac{n(n+1)}{2}$$
$$= C(n+1, 2)$$

Sum of Bitwise And of all subarray

$$a_1, a_2, .., a_n = \sum_{Bit} \sum_{Len} C(Len+1, 2) \times 2^{Bit}$$

Example. If for some bit b the array has runs of lengths $L_1, L_2, \ldots,$ that bit's contribution is

$$2^b \sum_i \frac{L_i(L_i+1)}{2}.$$

Sum of Bitwise AND of all subsequence

$$a_1, a_2, .., a_n = \sum_{Bit} (2^{Frequncy_{Bit}} - 1) \times 2^{Bit}$$

Sum of Bitwise OR of all subarray

$$a_1, a_2, .., a_n = \sum_{Bit} (C(n+1, 2) \times 2^{Bit}$$
$$- \sum_{L_{0subarray}} C(L+1, 2) \times 2^{Bit})$$

(7.11) Tree

All possible subtree path cost:

$$ans = \sum_i cost_i \times childCnt_i \times (n - childCnt_i)$$

(7.12) Pigeonhole Principle

- **General:** Putting n items in m boxes means at least one box gets $\lceil n/m \rceil = (n + m - 1)/m$ items.
- **Divisible Subarray:** Any array of size n ALWAYS has a subarray whose sum is divisible by n (since $n+1$ prefix sums mod n must repeat).
- **Mod of Pair Diffs:** Product of $|a_i - a_j| \bmod m$ for all pairs is ALWAYS 0 if $n > m$ (two items share mod m). If $n \leq m$, just brute force in $\mathcal{O}(n^2)$.
- **Pair Sum/XOR Repeat:** If pair sum/XOR is at most S, checking $S+1$ pairs guarantees a duplicate value. Break nested $\mathcal{O}(n^2)$ loop early!
- **4-Element XOR = 0:** If adjacent items differ by 1 bit (64-bit ints) and $n \geq 130$ (65 pairs), 4 items with XOR sum 0 ALWAYS exist.
- **Triangle Exists:** Array has 3 sides forming a triangle ($a + b > c$) if $n \geq 47$ (for $a_i \leq 10^9$) or $n \geq 90$ (for $a_i \leq 10^{18}$), due to Fibonacci growth.
- **Integer Midpoint:** Any 5 points in 2D (9 in 3D) with integer coords ALWAYS have a pair whose midpoint is also an integer (same odd/even parity).

(7.13) nCr, nPr Calculate

include modular arithmatic (Mint) template

```
vector<mint> fact(mxN), invF(mxN);
void precompute () {
    fact[0] = fact[1] = 1;
    for (int i = 2; i < mxN; ++i)
        fact[i] = fact[i-1] * i;
    invF[mxN-1] = fact[mxN-1].inv();
    for (int i = mxN-2; i >= 0; --i)
        invF[i] = mint(i+1)*invF[i+1];
}
mint nCr (int n, int r) {
    if (r < 0 || r > n) return 0;
    return fact[n]*invF[r]*invF[n-r];
}
mint nPr(int n, int r) {
    if (r < 0 || r > n) return 0;
    return fact[n] * invF[n - r];
}
```

(7.14) Catalan Numbers and Balanced Sequences

Catalan numbers count various balanced structures (e.g., valid parentheses sequences).

Recursive Formula:

$$C_0 = 1, \quad C_1 = 1, \quad C_n = \sum_{k=0}^{n-1} C_k C_{n-1-k}$$

Use: Count valid expressions, bracket sequences, tree shapes, etc.

Closed Form: Number of balanced bracket sequences of length $2n$:

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \frac{(2n)!}{(n+1)! \cdot n!}$$

Use: Count number of balanced bracket sequences of length $2n$.

With k bracket types:

$$\text{Count} = C_n \cdot k^n$$

Use: Add color/type variety to bracket problems.

Prefix Condition: Number of sequences with $n + k$ pairs and first k open brackets:

$$C_{n,k} = \frac{k+1}{n+k+1}\binom{2n+k}{n}$$

Use: Count bracket sequences with fixed number of opening brackets at start.

(7.15) Binomial Coefficients

$$\binom{n}{k} = \frac{n!}{k!\,(n-k)!}$$

Meaning: Number of ways to choose k items from n without regard to order.
Use: Appears in combinatorics, binomial expansion, Pascal's triangle, and counting problems.
Pascal's Identity:

$$\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$$

Used for recursive computation and building Pascal's Triangle.
Row Sum Identity:

$$\sum_{k=0}^{n} \binom{n}{k} = 2^n \quad \text{and} \quad \sum_{k=1}^{n} \binom{n}{k} = 2^n - 1$$

Total number of subsets (and non-empty subsets) of an n-element set.

Pascal's Triangle (First 6 Rows):

```
Start from Row 0 and Col 0,
[binom(r, c)]:
          1          - binom(0,0)
        1   1        - (1,0) (1,1)
      1   2   1      - (2,0) (2,1), ...
    1   3   3   1
  1   4   6   4   1
1   5  10  10   5   1
```

(7.16) Stirling Numbers

(7.16.1) First Kind ($s(n,k)$)

Counts permutations of n elements with k disjoint cycles:

$$s(n,k) = (n-1)\cdot s(n-1,k) + s(n-1,k-1)$$

Base cases:

$$s(0,0) = 1, \quad s(n,0) = s(0,n) = 0$$

Use: Helpful in permutation-based DP.

$$\sum_{k=0}^{n} s(n,k) = n!$$

Use: Total permutations of n items.
Example: $s(3,2) = 3$ — Three ways to arrange 3 elements into 2 cycles: (1)(23), (2)(13), (3)(12).

(7.16.2) Second Kind ($S(n,k)$)

Counts ways to partition n objects into k non-empty subsets:

$$S(n,k) = k \cdot S(n-1,k) + S(n-1,k-1)$$

Base cases:

$$S(0,0) = 1, \quad S(n,0) = S(0,n) = 0$$

Use: Useful in grouping problems.
Special case:

$$S(n,2) = 2^{n-1} - 1$$

Use: Special case for partitioning into two subsets.
Example: Counts ways to divide 3 elements into 2 non-empty subsets. $S(3,2) = 3$ — 1,23, 1,32, 2,31.
Coloring Interpretation:

$S(n,k)\cdot k!$ = Number of colorings using k colors such that each is used at least once

Use: Coloring or assignment problems ensuring coverage.

(7.16.3) r-associated Stirling Number

Number of partitions where each subset has at least r elements:

$$S_r(n,k) = \text{(custom recurrence)}$$

Use: Partition into subsets with at least r elements each. Useful in constrained grouping.

(7.16.4) Reduced Stirling Number $S_d(n,k)$

Partitions where each element in a subset differs by at least d:

$$S_d(n,k) = S(n-d+1, k-d+1) \quad (\text{when } n \geq k \geq d)$$

Use: Partition such that elements in each subset are at least distance d apart.

(8) Probability / Expected Value

(8.1) Probability basics

If an event E occurs in r ways out of n equally likely outcomes,

$$P(E) = \frac{r}{n}, \qquad P(E') = 1 - P(E).$$

Complement rule:

$$P(E) + P(E') = 1.$$

(8.1.1) Inclusion-Exclusion:

Two events A, B:

$$P(A \cup B) = P(A) + P(B) - P(A \cap B),$$
$$P(A \cap B) = P(B)\,P(A \mid B).$$

Three events A, B, C:

$$P(A \cup B \cup C) = P(A) + P(B) + P(C) - P(A \cap B) - P(B \cap C) - P(A \cap C) + P(A \cap B \cap C).$$

- **Notes:** If events are pairwise disjoint, all intersections vanish and the union probability is the sum of individual probabilities.

Conditional probability:

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)} \quad (P(B) > 0).$$

Independence:

$$P(A \cap B) = P(A)P(B).$$

(8.2) Expected value

Definition (discrete). If a random variable X takes values $x_1, \ldots, x_k$ with probabilities $p_1, \ldots, p_k$ then

$$\mathbb{E}[X] = \sum_{i=1}^{k} x_i p_i.$$

For a countable set of outcomes,

$$\mathbb{E}[X] = \sum_{i=1}^{\infty} x_i p_i,$$

provided the sum converges (absolutely in most uses).

Linearity of expectation (always true). For any random variables X, Y (not necessarily independent) and constants a, b,

$$\mathbb{E}[X+Y] = \mathbb{E}[X] + \mathbb{E}[Y],$$
$$\mathbb{E}[aX+b] = a\mathbb{E}[X] + b.$$

Indicator variables. If I_A is the indicator of event A (i.e. $I_A = 1$ when A occurs, else 0), then

$$\mathbb{E}[I_A] = P(A).$$

Use indicator variables plus linearity to simplify many counting expectations:

$$\mathbb{E}\Big[\sum_i I_i\Big] = \sum_i \mathbb{E}[I_i] = \sum_i P(I_i = 1).$$

Small examples

Fair die If X is the outcome of a fair six-sided die,

$$\mathbb{E}[X] = \frac{1+2+3+4+5+6}{6} = 3.5.$$

Expected number of fixed points in a random permutation of n Let I_j indicate that element j is a fixed point. Then $P(I_j = 1) = 1/n$, so

$$\mathbb{E}[\#\text{ fixed points}] = \sum_{j=1}^{n} \mathbb{E}[I_j] = n \cdot \frac{1}{n} = 1.$$

(8.2.1) Geometric / recursion

Example: "toss until first head" (success prob. p). Let T be the number of tosses until the first success (so $P(T=k) = (1-p)^{k-1}p$ for $k \geq 1$). By summing the series,

$$\mathbb{E}[T] = \sum_{k=1}^{\infty} k(1-p)^{k-1}p = \frac{1}{p}.$$

Equivalently, recursion: conditioning on the first toss gives

$$\mathbb{E}[T] = 1\cdot p + (1+\mathbb{E}[T])\cdot(1-p) \implies \mathbb{E}[T] = \frac{1}{p}.$$

For $p = \frac{1}{2}$ this yields $\mathbb{E}[T] = 2$.

(8.2.2) Recursive expectations

Law of total expectation. If after the first step one of several actions i happens with probability p_i, giving immediate reward r_i and leaving a subproblem with expected remaining value E_i, then

$$\mathbb{E}[X] \;=\; \sum_i p_i\,(r_i + E_i).$$

Solve the resulting linear equation(s) for the unknown expectation(s).

Example 1:"toss until first head" (success prob. p). Let T be the number of tosses until the first head. Condition on the first toss:

$$T = \begin{cases} 1, & \text{w.p. } p, \\ 1 + T', & \text{w.p. } 1-p, \end{cases}$$

where $T' \overset{d}{=} T$ (fresh copy). Taking expectations:

$$\mathbb{E}[T] = p\cdot 1 + (1-p)\cdot(1+\mathbb{E}[T]).$$

Solve for $\mathbb{E}[T]$:

$$\begin{aligned}\mathbb{E}[T] &= p + (1-p) + (1-p)\mathbb{E}[T] \\ &= 1 + (1-p)\mathbb{E}[T] = p\,\mathbb{E}[T] = 1 \\ \mathbb{E}[T] &= \frac{1}{p}.\end{aligned}$$

Example 2: expected number of fair–coin flips to see two consecutive heads. Use states: E_0 = expected flips starting with no previous head, E_1 = expected flips given the previous flip was a head.

$$E_0 = 1 + \frac{1}{2}E_1 + \frac{1}{2}E_0,$$
$$E_1 = 1 + \frac{1}{2}\cdot 0 + \frac{1}{2}E_0.$$

Solve the system. From the first equation $E_0 - \frac{1}{2}E_0 = 1 + \frac{1}{2}E_1$, so $E_0 = 2 + E_1$. Substitute into the second: $E_1 = 1 + \frac{1}{2}(2+E_1) = 2 + \frac{1}{2}E_1$, hence $\frac{1}{2}E_1 = 2$ and $E_1 = 4$. Thus $E_0 = 2 + 4 = 6$. So the expected number of flips to get two consecutive heads is 6.

(8.3) Variance (brief)

$$\mathrm{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2.$$

Affine property:

$$\mathrm{Var}(aX + b) = a^2\,\mathrm{Var}(X).$$

(8.3.1) Useful identities

Law of total probability:

$$P(A) = P(A \mid B)P(B) + P(A \mid B^c)P(B^c).$$

Bayes' theorem:

$$P(B \mid A) = \frac{P(A \mid B)P(B)}{P(A)} \quad (P(A) > 0).$$

Mismatch in permutation:

$$P(a_1) = 1 - \frac{1}{n}.$$
$$P(a_1..a_n) = n - 1$$

Number of connected component in a tree:

$$\begin{aligned}\mathbb{E}(x) &= \mathbb{E}[notEdge] + 1. \\ &= \mathbb{E}[notEdge]_1 + \mathbb{E}[notEdge]_2 ... + 1.\end{aligned}$$

(8.4) Contest tips

- Use inclusion–exclusion for unions of many events.
- Use Stirling numbers when counting set partitions (unlabeled boxes).
- Use indicator variables + linearity to turn combinatorial counts into simple expectations.
- For recursive expectations, condition on the first step and solve the resulting linear equation.

(9) Fibonacci Numbers

(9.1) Definition:

$$F_1 = F_2 = 1, \quad F_{n+1} = F_n + F_{n-1}$$

(9.2) Properties

- For a prime number p, the Fibonacci number F_p satisfies:
$$F_p \equiv \left(\frac{p}{5}\right) \pmod p,$$
where $\left(\frac{p}{5}\right)$ is the **Legendre symbol**, which indicates whether 5 is a quadratic residue modulo p.
- The greatest common divisor (GCD) of two Fibonacci numbers equals the Fibonacci number indexed by their GCD:
$$\gcd(F_m, F_n) = F_{\gcd(m,n)}.$$
- Consecutive and nearby Fibonacci numbers are coprime:
$$\gcd(F_n, F_{n+1}) = 1, \quad \gcd(F_n, F_{n+2}) = 1, \quad \gcd(F_{n+1}, F_{n+2}) = 1.$$
- The Fibonacci sequence modulo n is periodic, known as the **Pisano period**. The period length satisfies:
$$\text{period length} \leq 6n.$$
This helps to efficiently compute large Fibonacci numbers modulo n.
- A number n is Fibonacci if and only if either
$$5n^2+4 \quad \text{or} \quad 5n^2-4 \quad \text{(is a perfect square)}.$$
- Every third Fibonacci number is even:
$$F_{3k} \text{ is even.}$$
More generally, every k-th Fibonacci number is divisible by F_k:
$$F_k \mid F_m \quad \text{if} \quad k \mid m.$$

(9.3) Closed Form (Binet's Formula)

The n-th Fibonacci number can be calculated directly using the **closed-form expression**, known as Binet's formula:

$$F_n = \frac{1}{\sqrt{5}}\left(\left(\frac{1+\sqrt{5}}{2}\right)^n - \left(\frac{1-\sqrt{5}}{2}\right)^n\right).$$

Use: This formula helps compute Fibonacci numbers in $O(1)$ time without recursion or loops, useful when n is not too large (due to floating point precision).

(9.4) Sum of the First n Fibonacci Numbers

The sum of the first n Fibonacci numbers satisfies:

$$\sum_{i=1}^{n} F_i = F_{n+2} - 1.$$

(9.5) Log-factorial precompute

If answer fits within Integer: Compute $\log(n!)$ efficiently for many queries by precomputing prefix sums of logs:

$$\log(n!) = \sum_{k=1}^{n} \log k, \qquad \log(0!) = 0.$$

```cpp
const int MAXN = 1000000;
vector<double> logFact(MAXN+1);
void pre(int N){
    logFact[0] = 0.0;
    for(double i = 1; i <= N; ++i){
        logFact[i] = logFact[i-1] + log
            (i);
    }
}
// log(a*b) = log(a) + log(b)
// log(a/b) = log(a) - log(b)
// reverse = exp(i) or expl(i)
```

(10) Calendar Day

(10.1) Leap Year:

- (Year%400==0 or (Year%4==0 and Year%100!=0))

Zeller's congruence is an algorithm to compute the day of the week for any date. It has different versions for the Gregorian and Julian calendars.

(10.2) Gregorian Calendar

$$h = (q + \left\lfloor \frac{13(m+1)}{5} \right\rfloor + k + \left\lfloor \frac{k}{4} \right\rfloor + \left\lfloor \frac{j}{4} \right\rfloor - 2j) \bmod 7$$

(10.3) Julian Calendar

$$h = \left(q + \left\lfloor \frac{13(m+1)}{5} \right\rfloor + k + \left\lfloor \frac{k}{4} \right\rfloor + 5 - j\right) \bmod 7$$

(10.3.1) Where:

- h is the day of the week (0 = Saturday, 1 = Sunday, ..., 6 = Friday)
- q is the day of the month
- m is the month (March = 3, ..., January = 13, February = 14 of the **previous year**)
- k = year mod 100 (the year within the century)
- $j = \left\lfloor \frac{\text{year}}{100} \right\rfloor$ (the zero-based century)

(10.4) Adjustment for January and February

If the month is January or February, use:

$$m = 13 \text{ (January)}, \quad m = 14 \text{ (February)}, \quad \text{year} = \text{year} - 1$$

Use: Quickly determine the day of the week for a given date.)

(11) Number Theory

(11.1) Modular Arithmetic

```cpp
const ll M = 1e9+7, mxN = 1e5+9;
template<ll M>
struct Mint {
    ll a;
    Mint(ll _a = 0) : a((_a%M+M)%M){}
    Mint binExp(ll b) const {
        Mint ans = 1, z = a;
        for (; b; b >>= 1, z *= z)
            if (b & 1) ans *= z;
        return ans;
    }
    Mint inv() const { return binExp(M
        - 2); }
    Mint operator-() const { return
        Mint(-a); }
    Mint operator+(Mint o) const {
        return a + o.a; }
    Mint operator-(Mint o) const {
        return a - o.a + M; }
    Mint operator*(Mint o) const {
        return a * o.a; }
    Mint operator/(Mint o) const {
        return a * o.inv().a; }

    Mint& operator+=(Mint o) { return *
        this = *this + o; }
    Mint& operator-=(Mint o) { return *
        this = *this - o; }
    Mint& operator*=(Mint o) { return *
        this = *this * o; }
    Mint& operator/=(Mint o) { return *
        this = *this / o; }

    bool operator==(Mint o) const {
        return a == o.a; }
    bool operator!=(Mint o) const {
        return a != o.a; }
    bool operator<(Mint o) const {
        return a < o.a; }

    friend ostream& operator<<(ostream&
        os, Mint m) {
        return os << m.a;
    }
    friend istream& operator>>(istream&
        is, Mint& m) {
        ll x; is >> x; m = Mint(x);
        return is;
    }
};
// usage:
// using mint = Mint<M>;
// mint x=2, y=3, x.pow(y.a), x.inv()
```

(11.2) Matrix multiplication

Multiply matrices A (n x p) and B (p x m) and return C = A*B (n x m).

Recurrence Example 1:

$f(n) = af(n-1) + bf(n-2) + f(n-3) + C$

Vector form:

$$F(n) = \begin{bmatrix} f(n) \\ f(n-1) \\ f(n-2) \\ 1 \end{bmatrix}, \quad M = \begin{bmatrix} a & b & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Matrix relation:

$$F(n) = M^{n-2} \cdot F(2), \quad F(2) = \begin{bmatrix} f(2) \\ f(1) \\ f(0) \\ C \end{bmatrix}$$

Recurrence Example 2:

$a_i = (c_1 a_{i-1} + c_2 a_{i-2} + \cdots + c_n a_{i-n}) + p + iq + i^2 r.$

Step 1: Express i and i^2 in terms of $(i-1)$:

$$\begin{aligned} \mathbf{i} &= (i-1) + 1 \\ \mathbf{i^2} &= ((i-1)+1)^2 \\ &= (i-1)^2 + 2(i-1) + 1 \end{aligned}$$

Step 2: Non-homogeneous part:

$$\begin{aligned} \mathbf{Y_i} &= p + iq + i^2 r. \\ &= p + q((i-1)+1) \\ &\quad + r((i-1)^2 + 2(i-1) + 1). \\ &= p + q(i-1) + q \\ &\quad + r(i-1)^2 + 2r(i-1) + r \\ &= (p + r + q) + (q + 2r)(i-1) + r(i-1)^2 \end{aligned}$$

Step 3: Full recurrence:

$$\begin{aligned} a_i = {} & c_1 a_{i-1} + c_2 a_{i-2} + \cdots + c_n a_{i-n} \\ & + (p + q + r) \\ & + (q + 2r)(i-1) \\ & + r(i-1)^2 \end{aligned}$$

Step 4: Vector form:

$$F(i) = \begin{bmatrix} a_i \\ a_{i-1} \\ \vdots \\ a_{i-n+1} \\ 1 \\ i \\ i^2 \end{bmatrix}$$

$$\mathbf{U} = p + q + r, \quad \mathbf{V} = q + 2r$$

Step 5: Augmented matrix:

$$M = \begin{bmatrix} c_1 & c_2 & \dots & c_n & \mathbf{U} & \mathbf{V} & r \\ 1 & 0 & \dots & 0 & 0 & 0 & 0 \\ 0 & 1 & \dots & 0 & 0 & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & \dots & 1 & 0 & 0 & 0 \\ 0 & 0 & \dots & 0 & 1 & 1 & 0 \\ 0 & 0 & \dots & 0 & 1 & 2 & 1 \end{bmatrix}$$

Step 6: Matrix Relation:

$$F(i) = M^{i-n} \cdot F(n), \quad F(n) = \begin{bmatrix} a_n \\ a_{n-1} \\ \vdots \\ a_1 \\ 1 \\ n \\ n^2 \end{bmatrix}$$

```cpp
#define vvl vector<vector<ll>>
const int mod = 1e9 + 7;
const ll inf = 1e18;
vvl matExp(const vvl &a, const vvl &b)
    {
    int n = a.size();
    int m = a[0].size();
    int k = b[0].size();
```

```
    vvl ans(n, vector<ll>(m, 0));
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < m; ++j) {
            for (int via = 0; via < k;
                 ++via) {
                ans[i][j] += (a[i][via]
                     * b[via][j]);
                ans[i][j] %= mod;
            }
        }
    }
    return ans;
}

struct Matrix {
    vvl mat;
} t[35];

void Din() {
    int n, m, q; cin >> n >> m >> q;
    vvl g(n, vector<ll> (n, 0));
    for (int i = 0; i < m; ++i) {
        int x, y; cin >> x >> y;
        x -= 1, y -= 1; g[x][y] = 1;
    }
    t[0].mat = g;
    for (int i = 1; i < 32; ++i) {
        t[i].mat = matExp(t[i - 1].mat,
             t[i - 1].mat);
    }
    while (q--) {
        int s, e, k;
        cin >> s >> e >> k;
        s -= 1, e -= 1;
        vvl ans(1, vector<ll> (n, 0));
        ans[0][s] = 1;
        for (int i = 0; i < 31; ++i) {
            if ((k >> i) & 1) {
                ans = matExp(ans, t[i].
                    mat);
            }
        }
        cout << ans[0][e] << '\n';
    }
}
```

$(AdjMatrix)^n$ finds count of path that has exactly n edges.

Notes:

- Time complexity: $O(n \cdot p \cdot m)$.

(11.3) GCD & Diophantine

- $\gcd(a,b) = \gcd(a-b,b) \quad (a \geq b)$
- $\gcd(a,b) = __gcd(a,b)$
- $\text{lcm}(a,b) = \frac{|ab|}{\gcd(a,b)}$
- If $\gcd(a,b) = 1$, then $\text{lcm}(a,b) = |ab|$.
- $ax + by = c$ has integer solutions iff $g = \gcd(a,b) \mid c$.
- Given one solution (x_0, y_0), all solutions:

$$x = x_0 + \frac{b}{g}t, \quad y = y_0 - \frac{a}{g}t, \quad t \in \mathbb{Z}.$$

(11.4) Cayley's Formula

Cayley's formula states that the number of labeled trees with n vertices is:

$$n^{n-2}$$

A **labeled tree** is an undirected tree where each vertex is uniquely labeled from 1 to n.

- For $n = 3$, total trees $= 3^{3-2} = 3$
- For $n = 4$, total trees $= 4^{4-2} = 16$

(11.5) Binary Exponential

```
/**
 * Time Complexity: O(log b)
 * Computes (a^b) % M
 * Calculate(1e18^1e18) without uint
 * inv(a) = a^(M-2) % M
 */
ll binMul(ll a, ll b, ll M) {
    ll ans = 0;
    while (b) {
        if (b & 1) ans = (ans + a) % M;
        a = (a + a) % M; b >>= 1;
    }
    return ans % M;
    // return ((__int128) a * b) % M;
}
ll binExp(ll a, ll b, ll M) {
    ll ans = 1;
    while (b) {
        if (b & 1) ans = binMul(ans, a, M
            );
        a = binMul(a, a, M); b >>= 1;
    }
    return ans;
}
// Usage:
// binExp(a, M - 2, M); // a^(-1)
// binExp(a, b, M); // a^b
```

(11.6) Extended Euclidean GCD

```
/**
 * Time Complexity: O(log(min(a, b)))
 * Computes gcd of a and b, and finds x
     , y such that: ax + by = gcd(a, b)
 * Useful to find modular inverse when
     gcd(a, m) = 1.
 */
ll exGCD(ll a, ll b, ll &x, ll &y) {
    if (!b) return x = 1, y = 0, a;
    ll gcd = exGCD(b, a % b, y, x);
    return y -= a / b * x, gcd;
}
ll floorDiv(ll a, ll b) {// for maxK
    assert(b != 0);
    return a/b-((a^b)<0 && a%b);
}
ll ceilDiv(ll a, ll b) {// for minK
    assert(b != 0);
    return a/b+((a^b)>0 && a%b);
}
void Din() {
    ll n, m, a, k;
    while (cin >> n >> m >> a >> k) {
        if (!n && !m && !a && !k)
            return;
        // need fing T >= max(n, k+a)
        // let m jump x times, a jump y
            times
        // x >= 0, y >= 1
        // T == n(mod m), T == k(mod a)
        // Linear Diophantine Equation
        // n + m*x = k + a*y
        // => m*x - a*y = k - n, -->
            Form ax + by = c <--
        ll A = m, B = -a, C = k-n;
        ll X = 0, Y = 0;
        ll g = exGcd(abs(A), abs(B), X,
            Y);
        if (C % g != 0) {
            cout << "Impossible\n";
            continue;
        }
        X *= (C/g), Y *= (C/g);
        if (A < 0) X = -X;
        if (B < 0) Y = -Y;
        // dbg(X, Y);
        ll dx = a / g, dy = m / g;
        ll minX = ceilDiv(-X, dx); // X
            +dx*k >= 0
        ll minY = ceilDiv(1-Y, dy); //
            Y+dy*K >= 1
        // ll maxY = floorDiv(1-Y, dy);
        // if (minX > maxY) {// find X
            >0 && Y>0
        //     cout << "Impossible\n";
        //     continue;
        // }
        ll kk = max(minX, minY);
        X += dx*kk, Y += dy*kk;
        if (X < 0 || Y < 0) {
            cout << "Impossible\n";
            continue;
        }
        // cout << X << " " << Y << "\n
            ";
        ll ans = n + X*m;
        cout << ans << "\n";
    }
}
```

(11.7) Basis Vector

```
template<class T, int m = 31>
struct vecBasis {
    vector<T> basis; T b_size = 0;
    vecBasis() : basis(m, 0) {}
    void insert(T x) {
        for (int i = m - 1; ~i; --i) {
            if ((x >> i) & 1) {
                if (basis[i])
                    x ^= basis[i];
                else {
                    b_size += 1;
                    basis[i] = x; break
                        ;
                }
            }
        }
    }
    bool can(T x) { // Xor sum = x
        for (int i = m - 1; ~i; --i) {
            x = min(x, x ^ basis[i]);
        }
        return x == 0;
    }
    T getMax() {
        T ans = 0;
        for (int i = m - 1; ~i; --i) {
            ans = max(ans, ans ^ basis[
                i]);
        } return ans;
    }
    T uniqueSubset() {
        return (1ll << b_size);
    }
};
// Usage...
vecBasis<ll, 63> b;
b.insert(x); b.getMax()
```

(11.8) Mobius + Mertens

```
/**
 * Computes primes up to N using sieve
     in O(N log log N).
 * Computes for each number:
 * - smallest prime factor (spf),
 */
const int mxN = 50000100;
vector<int> spf(mxN), mob(mxN),
    prefixMob(mxN);
void precompute () {
    iota(all(spf), 0);
    for (int i = 2; i * i < mxN; ++i) {
        if (spf[i] == i) {
            for (int j = i*i; j < mxN;
                j += i) {
                spf[j] = spf[i];
            }
        }
    }
    mob[1] = 1, prefixMob[1] = 1;
    for (int i = 2; i < mxN; ++i) {
        if (spf[i] == i) mob[i] = -1;
        else {
            int p = spf[i], x = i / p;
            if (x % p == 0) mob[i] = 0;
            else mob[i] = -mob[x];
        }
        prefixMob[i] = prefixMob[i-1] +
            mob[i];
    }
}
unordered_map<ll, ll> dp;
int getMertens (ll n) {
    if (n < mxN) return prefixMob[n];
    if (dp.count(n)) return dp[n];
    ll ans = 1;
    for (ll i = 2, nxt; i <= n; i = nxt
        + 1) {
        // 5/3 = 1, 5/1 = 5, range [3,
            5] cover same mobious[q]
        ll q = n / i, nxt = n / q;
        ans -= (nxt - i + 1) *
            getMertens(q);
    }
    return dp[n] = ans;
}

void Din() {
    ll n;
    cin >> n;
    ll sq = sqrtl(n);
    ll ans = 0, last = 0;
    for (ll i = 1, nxt; i <= sq; i =
        nxt + 1) {
        // same like getMertens but
            cover range for square
        ll val = n / (i*i);
        ll nxt_sq = sqrtl(n/val);
        nxt = min(sq, nxt_sq);
        ll curr = getMertens(nxt);
        ans += (curr - last) * val; //
            inclusion/exclusion
        last = curr;
    }
    cout << ans << '\n';
}
```

(11.9) Segmented Sieve

```
/**
 * Find primes in range [L, R] in O((R-
     L+1) * sqrt(R))
 */
void Din() {
    sieve(); // precalculate
    int lb, rb;
    cin >> lb >> rb;
    vector<int> primes;
    for (int i = 2; i * i <= rb; ++i) {
        if (isPrime[i]) primes.pb(i);
    }
    vector<bool> ans(rb - lb + 1, 1);
    for (auto &i : primes) {
        int k = (lb / i) * i; // First
            Multiple of J......
        if (k < lb) k += i;
        for (int j = max(k, i * i); j
            <= rb; j += i) {
            ans[j - lb] = 0;
        }
    }
    for (int i = max(lb, 2); i <= rb;
        ++i) {
        if (ans[i-lb]) cout<<i<<'\n';
    } cout << '\n';
}
```

(11.10) Pollard Rho

include millerRabin.cpp

```
/**
 * First pre-calculate SPF and primes
 * Factorize of a number <= 1e18.
 * Time: O(N^(1/4) * O(logN^3)[miller].
 */
ll add_mod(ll a, ll b, ll mod) {
    return ((a += b) >= mod ? a - mod :
        a);
}
ll mul_mod(ll a, ll b, ll mod) {
    ll ans = ((__int128) a * b) % mod;
    return ans;
}
ll pollardRho(ll n) {
    mt19937 rnd(chrono::steady_clock::
        now().time_since_epoch().count
        ());
    ll x = rnd() % n, y = x, c = rnd()
        % n; // also fine [0, 0, 1]
    ll step = 30, prd = 2, q;
    auto f = [&](ll a)->ll {
        return add_mod(mul_mod(a, a, n)
            , c, n);
    };
    while (step++ % 40 || __gcd(prd, n)
        == 1) {
        if (x == y) {x = ++c; y = f(x)
            ;}
        q = mul_mod(prd, abs(x - y), n)
            ;
        if (q) prd = q;
        x = f(x), y = f(f(y));
    }
    return __gcd(prd, n);
}
vector<ll> factorize(ll n) {
    if (n == 1) return {};
    if (miller(n)) return {n};
    vector<ll> ans, add;
    while (n > 1 && n < mxN) {
        ans.pb(spf[n]); n /= spf[n];
    }
    if (n >= mxN) {
        ll x = pollardRho(n);
        ans = factorize(x);
        add = factorize(n / x);
        ans.insert(ans.end(), all(add))
            ;
    }
    return ans;
}
// Usage:
// auto ans = factorize(n); sort(all(
    ans));
```

(11.11) Miller Rabin

```
/**
 * Time Complexity: O(log n)
In miller rabin, it holds,
    a^d = 1 (mod N) or,
    a^(2^s*d) = n - 1 (mod n)
Used for prime number check upto 7e18.
 */
bool millerRabin(ll n) {
    if (n == 2 || n == 3) return 1;
    if (n <= 1 || n % 2 == 0) return 0;
    ll d = n - 1; // need [ n-1 = a^s*d
        1<a<n-1, d is Odd ]
    while (d % 2 == 0) {d >>= 1;}
    vector<ll> pfs = {2, 3, 5, 7, 11,
        13, 29, 37, 43};
    for (auto &x : pfs) {
        if (x >= n) return 1;
        ll curr_d = d;
        ll mod = binExp(x, curr_d, n);
        if (mod != 1) {
            while (curr_d != n - 1 &&
                mod != n - 1) {
                mod = binMul(mod, mod,
                    n);
                curr_d *= 2;
            }
            if (mod != n - 1) return 0;
        }
    }
    return 1;
}
```

(11.12) FFT

```
struct Cx {
dbl a, b;
Cx (dbl _a = 0, dbl _b = 0) : a(_a), b(
    _b) {}
Cx operator+(Cx o) const { return Cx(a+
    o.a, b+o.b); }
Cx operator-(Cx o) const { return Cx(a-
    o.a, b-o.b); }
Cx operator*(Cx o) const { return Cx(a*
    o.a-b*o.b, a*o.b+b*o.a); }
};
void FFT (vector<Cx> &p, bool inv=0) {
    int n = sz(p);
    static vector<Cx> rt(2, Cx(1));
    for (int z = sz(rt); z<n; z *= 2) {
        rt.resize(2*z);
        for (int j = 0; j < z; ++j) {
            ld g = PI*j/z;
            rt[z+j] = Cx(cosl(g), sinl(
                g));
        }
    }
    for (int i = 1, j = 0; i<n; ++i){
        int b = n >> 1;
        for (; j & b; b >>= 1) j ^= b;
        j ^= b;
        if (i < j) swap(p[i], p[j]);
    }
    for (int z = 1; z<n; z *= 2) {
        for (int i = 0; i<n; i += 2*z){
```

```
            for (int j = 0; j<z; ++j){
                Cx t=rt[z+j]*p[i+j+z];
                p[i+j+z] = p[i+j]-t;
                p[i+j] = p[i+j]+t;
            }
        }
    }
    if (inv) {
        reverse(p.begin()+1, p.end());
        for (auto &x : p)
            x.a /= n, x.b /= n;
    }
}

template<class T>
vector<ll> mulFFT (vector<T> &a, vector
    <T> &b) {
    if (!sz(a) || !sz(b)) return {};
    int nm = sz(a)+sz(b)-1, n = 1;
    while (n < nm) n *= 2;
    vector<Cx> x(n);
    for (int i = 0; i < n; ++i) {
        if (i < sz(a)) x[i].a = a[i];
        if (i < sz(b)) x[i].b = b[i];
    }
    FFT(x);
    for (auto &c : x) c = c * c;
    FFT(x, 1);
    vector<ll> ans(nm);
    for (int i = 0; i < nm; ++i)
        ans[i] = llround(x[i].b / 2);
    return ans;
}
// Usage: vll a = {1, 2, 3}, b = {4,
    5};
// auto c = mulFFT(a, b); // exact
    integers (no mod), negatives OK
// c[i] = coefficients of x^i.
```

(11.13) NTT

include binExp.cpp

```
const int mod = 998244353;
// NTT<M>: M = c*2^k+1 prime
    (998244353: len<=2^23, 167772161:
    2^25, 469762049: 2^26), root R=3
template<int M, int R = 3> struct NTT {
// binExp() {...}
void run(vll &p, bool inv = 0){
    int n = sz(p);
    assert((M-1) % n == 0); // too big
    vll rt(2, 1);
    for (int z = sz(rt); z < n; z*=2) {
        rt.resize(2*z);
        ll g = binExp(R, (M-1)/ (2*z));
        rt[z] = 1;
        for (int k = 1; k < z; ++k)
            rt[z+k] = rt[z+k-1]*g%M;
    }
    for (int i = 1, j = 0; i < n; ++i){
        int b = n >> 1;
        for (; j & b; b>>=1) j ^= b;
        j ^= b;
        if (i < j) swap(p[i], p[j]);
    }
    for (int z = 1; z < n; z *= 2) {
        for (int i = 0; i < n; i += 2*z
            ) {
            for (int k = 0; k < z; ++k)
                {
                ll x = p[i+k], y = p[i+
                    k+z] * rt[z+k] % M;
                p[i+k] = x+y >= M ? x+y
                    -M : x+y;
                p[i+k+z] = x-y < 0 ? x-
                    y+M : x-y;
            }
        }
    }
    if (inv) {
        reverse(p.begin()+1, p.end());
        ll ni = binExp(n, M-2);
        for (auto &x : p) x = x*ni%M;
    }
}
vll mul(vll a, vll b) {
    if (!sz(a) || !sz(b)) return {};
    for (auto &x : a) x = (x%M+M)%M;
    for (auto &x : b) x = (x%M+M)%M;
    int nm = sz(a)+sz(b)-1, n = 1;
    while (n < nm) n *= 2;
    a.resize(n), b.resize(n);
    run(a), run(b);
    for (int i = 0; i < n; ++i)
        a[i] = a[i] * b[i] % M;
    run(a, 1); a.resize(nm);
    return a;
}
};
// Any modulus M (1e9+7, 1e9+9, ...), M
    <= ~2e9, |a|+|b|-1 <= 2^23. 3 NTTs
    + Garner CRT.
// 998244353, 167772161, 469762049
vll mulMod(vll a, vll b, ll M) {
    if (!sz(a) || !sz(b)) return {};
    for (auto &x : a) x = (x%M+M)%M;
    for (auto &x : b) x = (x%M+M)%M;
    const ll p1 = (119 << 23)+1, p2 =
        (5 << 25)+1, p3 = (7 << 26)+1;
    NTT<p1> x; NTT<p2> y; NTT<p3> z;
    auto r = x.mul(a, b), s = y.mul(a,
        b), t = z.mul(a, b);
    ll k1 = y.binExp(p1, p2-2), k2 = z.
        binExp(p1*p2%p3, p3-2), p12 =
        p1%M*(p2%M)%M;
    for (int i = 0; i < sz(r); ++i) {
        ll j1 = ((s[i]-r[i])%p2+p2) %
            p2 * k1 % p2;
        ll j2 = ((t[i]-r[i]-p1 % p3 *
            j1)%p3 + p3)%p3 * k2%p3;
        r[i] = (r[i]%M+p1%M * j1 + p12
            * j2) % M;
    }
    return r;
}
// usage:
// NTT<mod> t;
// auto d = t.mulNTT(a, b); // mod
    998244353
// auto e = mulMod(a, b, 1000000007);
    // any mod
```

(11.14) Euler's Totient (Phi Function)

If $\gcd(x, m) = 1$, $x^{\phi(m)} \equiv 1 \pmod{m}$

If $\gcd(x, m) = 1$, $x^n \equiv x^{n \bmod \phi(m)} \pmod{m}$

Eulerś Totient function $\phi(n)$ counts the number of integers from 1 to n that are co-prime with n.

1. Computing $\phi(1)$ to $\phi(n)$ using a sieve-like method:

```
// Time: O(N log log N)
void phi_1_to_n(int n) {
    vector<int> phi(n + 1);
    for (int i = 0; i <= n; i++)
        phi[i] = i;

    for (int i = 2; i <= n; i++) {
        if (phi[i] == i) {
            for (int j = i; j <= n; j
                += i)
                phi[j] -= phi[j] / i;
        }
    }
}
```

2. Computing $\phi(n)$ for a single number n in $O(\sqrt{n})$: This method factorizes n and applies the formula:

$$\phi(n) = n \times \prod_{p|n} \left(1 - \frac{1}{p}\right)$$

where p are the distinct prime factors of n.

```
// Time: O(sqrt(n))
int phi(int n) {
    int result = n;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            while (n % i == 0)
                n /= i;
            result -= result / i;  //
                Remove contribution of
                prime factor i
        }
    }
    if (n > 1)
        result -= result / n;  //
            Remove contribution of last
             prime factor
    return result;
}
```

(11.15) Count GCD(a, b) = X

```
/**
 * Time Complexity: O(n log n)
 * Usage: count gcd(x, y) = 1/2/3 etc.
 * Also can use for Lcm(x, y) = L with
     modification.
 */
void Process() {
    for (int i = 1; i < mxN; ++i) {
        for (int j = i; j < mxN; j += i
            ) {
            ans[i] += freq[j];
            // for LCM pair, ans[j] +=
                freq[i];
        }
    }
    for (int i = 1; i < mxN; ++i) {
        // way to choose 2 itmes, can
            be 3/4/5 etc.
        if (ans[i] < 2) ans[i] = 0;
        else ans[i] = nCr(ans[i], 2);
    }
    for (int i = mxN - 1; i; --i) {
        for (int j = 2 * i; j < mxN; j
            += i) {
            ans[i] -= ans[j];
        }
    }
```

```
}

/**
 * Usage example:
 * ans[6] = count of pair/triple etc.
     of gcd/lcm equal 6,
 */
```

(11.16) Count & SOD of N

```
/**
 * Calculates the count and sum of
     divisors of a number using its
     prime factorization.
 * Precompute spf using Sieve, and
     prime factors count
 * Time complexity:  O(log n)
 */
```

If $n = P_1^{n_1} \times P_2^{n_2} \times \cdots \times P_k^{n_k}$ then,

(11.16.1) Count of divisors:

$$\text{count} = (n_1+1)(n_2+1)\cdots(n_k+1)$$

(11.16.2) Sum of divisors:

$$\begin{aligned}\text{sum} &= \prod_{i=1}^{k} \frac{P_i^{n_i+1}-1}{P_i-1} \\ &= \left(1+P_1+P_1^2+\cdots+P_1^{n_1}\right) \times \cdots \\ &\quad \times \left(1+P_k+P_k^2+\cdots+P_k^{n_k}\right)\end{aligned}$$

Example: For $36 = 2^2 \times 3^2$,

$$\text{count} = (2+1)(2+1) = 9$$

$$\text{sum} = \frac{2^3-1}{2-1} \times \frac{3^3-1}{3-1} = 7 \times 13 = 91$$

(11.16.3) Product of divisors:

$$\begin{aligned}\text{product} &= N^{d/2} \\ &= (P_1^{n_1} \cdot P_2^{n_2} \cdots P_k^{n_k})^{\frac{1}{2}\prod_{i=1}^{k}(n_i+1)}\end{aligned}$$

If d is odd (Perfect Square):

$$\begin{aligned}\text{product} &= N^{\frac{d-1}{2}} \times \sqrt{N} \\ &= N^{\frac{d}{2}} \times \sqrt{N}^{\,d \bmod 2}\end{aligned}$$

(11.17) Sum of Divisors [1...N]

```
ll sumOfDivisors(ll N) {
    ll sum = 0;
    for (ll i = 1; i <= N; i++) {
        sum += i * (N / i);
    }
    return sum;
}
```

SOD of all numbers from 1 to $N = \sum_{i=1}^{N} i \cdot \left\lfloor \frac{N}{i} \right\rfloor$

Explanation: Every number i is a divisor of exactly $\left\lfloor \frac{N}{i} \right\rfloor$ numbers between 1 and N, so we add $i \times \left\lfloor \frac{N}{i} \right\rfloor$ to the total sum. **Example:** For $N = 2$,

$$\text{Sum} = 1 \cdot \left\lfloor \frac{2}{1} \right\rfloor + 2 \cdot \left\lfloor \frac{2}{2} \right\rfloor = 1 \cdot 2 + 2 \cdot 1 = 4$$

(11.18) Power of a Prime in $n!$

```
/**
 * This counts how many times p divides
     n!
 * Time: O(log_p n)
 */

ll largestPower(ll n, ll p) {
    if (n < 0 || p < 2) return 0;
    ll x = 0;
    while (n) {n /= p; x += n; }
    return x;
}
```

Formula:

$$e = \left\lfloor \frac{n}{p} \right\rfloor + \left\lfloor \frac{n}{p^2} \right\rfloor + \left\lfloor \frac{n}{p^3} \right\rfloor + \cdots$$

Example: Find the power of 5 in 100!

$$e = \left\lfloor \frac{100}{5} \right\rfloor + \left\lfloor \frac{100}{25} \right\rfloor + \left\lfloor \frac{100}{125} \right\rfloor = 20 + 4 + 0 = 24$$

Answer: The power of 5 in 100! is $\boxed{24}$.

(12) Graph

(12.1) Direction Arrays

```
// R D L U.....
int dx[4] = {0, 1, 0, -1};
int dy[4] = {1, 0, -1, 0};
int di[8] = {1, 1, 1, -1, -1, -1, 0,
    0};
int dj[8] = {1, -1, 0, 1, -1, 0, 1,
    -1};
int ki[8] = {1, 2, 1, 2, -1, -2, -1,
    -2};
int kj[8] = {2, 1, -2, -1, 2, 1, -2,
    -1};
```

(12.2) Dijkstra's

```
/**
 * Time Complexity: O((n + m) log n)
 * Computes the shortest distance from
     a single source node to all others
     .
 * Requires non-negative edge weights.
 */
void dijkstra(int src) {
    vector<ll> dist(n + 1, 1e9);
    dist[src] = 0;
    // Min-heap: (distance, node)
    priority_queue<pair<ll, ll>, vector
        <pair<ll, ll>>, greater<pair<ll
        , ll>>> pq;
    pq.push({0, src});
    while (!pq.empty()) {
        auto [dis, u]=pq.top(); pq.pop
            ();
        if (dis != dist[u]) continue;
        for (auto [v, w] : G[u]) { // G
            [u] holds pairs (v, weight)
            if (dist[v] > dist[u] + w)
                {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
}
```

(12.3) Floyd-Warshall

```
/**
 * Time Complexity: O(n^3)
 * It computes the shortest distance
     between every pair of nodes.
 * Works for positive and negative edge
     weights.
 * Does not work correctly if the graph
     contains a negative weight cycle.
 */
int n, m;
vector<vector<long long>> dist(n + 1,
    vector<long long>(n + 1, LLONG_MAX)
    );
for (int via = 1; via <= n; via++) {
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            if (dist[i][via] !=
                LLONG_MAX && dist[via][
                j] != LLONG_MAX) {
                dist[i][j] = min(dist[i
                    ][j], dist[i][via]
                    + dist[via][j]);
            }
        }
    }
}
```

(12.4) Bellman-Ford

```
/**
 * Time Complexity: O(n * m)
 * Detects the shortest or longest path
     from a source node.
 * Can detect the presence of a
     positive or negative cycle.
 * This version returns -1 if there's a
     positive cycle on the way to the
     destination.
 */
int n, m, src;
vector<vector<pair<ll, ll>>> graph;
vector<ll> dist(n + 1, LLONG_MIN);
dist[src] = 0;
// Relax edges n times to detect
    positive cycle
for (int cnt = 1; cnt <= n; cnt++) {
    for (int u = 1; u <= n; u++) {
        for (auto [w, v] : graph[u]) {
            if (dist[u] != LLONG_MIN &&
                dist[v] < dist[u] + w)
                {
                if (cnt == n) {
                    // Positive cycle
                        detected
                    return {-1};
                }
                dist[v] = dist[u] + w;
            }
        }
    }
}
```

(12.5) Kruskal's for MST

```
/**
 * include DSU first.
 * Time: O(E*logE)
 */
void Din() {
```

```
    int n, m;
    cin >> n >> m;
    DSU t(m);
    vector<arr<int, 3>> v(m+1);
    for (int i = 1; i <= m; ++i) {
        cin >> v[i][1] >> v[i][2] >> v[
            i][0];
    }
    sort(all(v)); ll sum = 0;
    for (int i = 1; i <= m; ++i) {
        int x = v[i][1], y = v[i][2];
        if (t.Find(x) != t.Find(y)) {
            sum += v[i][0]; cnt += 1;
            t.Union(x, y);
        }
    }
    cout << sum << '\n';
}
```

(12.6) SCC (Kosaraju's Algo)

```
/**
 * Time Complexity: O(n + m)
 * Finds SCCs in a directed graph.
 * First DFS on original graph to get
     finish order.
 * Second DFS on reversed graph to
     identify components.
 */
vector<int> g[mxN], g2[mxN], component[
    mxN], scc[mxN], root(mxN), ord;
bool vis[mxN];
void dfs(int curr) {
    vis[curr] = 1;
    for (auto &x : g[curr]) {
        if (!vis[x]) dfs(x);
    }
    ord.pb(curr);
}
void dfs2(int curr, int comp) {
    vis[curr] = 1; root[curr] = comp;
    component[comp].pb(curr);
    for (auto &x : g2[curr]) {
        if (!vis[x]) dfs2(x, comp);
    }
}
void solve() {
    int n, m; cin >> n >> m;
    for (int i = 0; i < m; ++i) {
        int x, y; cin >> x >> y;
        g[x].pb(y); // Main
        g2[y].pb(x); // Reverse
    }
    for (int i = 1; i <= n; ++i) {
        if (!vis[i]) dfs(i);
    }
    reverse(begin(ord), end(ord));
    memset(vis, 0, sizeof(vis));
    int comp = 1;
    for (auto &x : ord) {
        if (!vis[x]) dfs2(x, comp++);
    }
    for (int i = 1; i < n; ++i) {
        for (auto &x : g[i]) {
            if (root[x] ^ root[i]) {
            scc[root[i]].pb(root[x]);
            }
        }
    }
}
```

(12.7) HopCroft Matching

```
/**
 * Finds maximum number of disjoint
     pairs between two sets.
 * a[u] = matched right node of u
 * b[v] = matched left node of v
 */
vector<vector<int>> g;
vector<ll> a, b, d;
int n, m;
const ll inf = 1e18;
void reset() {
    g.assign(n+1,{}); a.assign(n+1, 0);
    b.assign(n+1,0); d.assign(n+1, 0);
}
bool bfs() {
    queue<int> q;
    for (int i = 1; i <= n; ++i) {
        if (!a[i]) {d[i]=0; q.push(i);}
        else { d[i] = inf; }
    }
    bool ok = 0;
    while (!q.empty()) {
        int node = q.front(); q.pop();
        for (auto &child : g[node]) {
            int v = b[child];
            if (v == 0) ok = 1;
            else if (d[v] == inf) {
                d[v] = d[node] + 1;
                q.push(v);
            }
        }
    }
    return ok;
}
bool dfs(int node) {
    for (auto &child : g[node]) {
        int v = b[child];
        if (v == 0 || (d[v] == d[node]
            + 1 && dfs(v))) {
            a[node] = child;
            b[child] = node;
            return 1;
        }
    }
    return d[node] = inf, 0;
}
int HopCroft() {
    int ans = 0;
    while (bfs()) {
        for (int i = 1; i <= n; ++i) {
            if (!a[i] && dfs(i)) {
                ans += 1;
            }
        }
    }
    return ans;
}
int cnt = HopCroft();
```

(12.8) DSU + RollbackDSU

```
/**
 * Time Complexity: O(log n)
 * Average: Amortized O(alpha(n)), per
     operation (almost constant <= 4)
 * Supports rollback to previous states
     .
*/
struct RollbackDSU {
vector<int> ps;
vector<pair<int, int>> st;
// negative for parent itself with size
    1..
RollbackDSU(int n) : ps(n+1, -1) {}
bool sameSet(int a, int b) {
    return Find(a) == Find(b);
}
int Find(int x) {
    return ps[x] < 0 ? x : Find(ps[x]);
    // return ps[x] < 0 ? x : ps[x] =
        Find(ps[x]); // only DSU
}
bool Union(int a, int b) {
    a = Find(a), b = Find(b);
    if (a == b) return false;
    if (ps[a] > ps[b]) swap(a, b);
    st.push_back({a, ps[a]}); // for
        rollback
    st.push_back({b, ps[b]}); // for
        rollback
    ps[a] += ps[b]; ps[b] = a;
    return true;
}
int getSize(int x) {
    return -ps[Find(x)];
}
}
// Use only for rollback
int time() { return st.size(); }
void rollback(int t) {
    for (int i = time(); i-- > t;)
        ps[st[i].first] = st[i].second;
    st.resize(t);
}
};
```

(12.9) Re-rooting Tree DP

```
// Ex: Sum of dist to all nodes
// 1. Get subtree info (root = 1)
void dfs1(int u, int p) {
   sz[u] = 1, dp[u] = 0;
   for (int v : adj[u]) if (v != p) {
      dfs1(v, u);
      sz[u]+=sz[v]; dp[u]+=dp[v]+sz[v];
   }
}
// 2. Move root u -> v & backtrack
void dfs2(int u, int p) {
   ans[u] = dp[u]; // ans for root u
   for (int v : adj[u]) if (v != p) {
      // a. Save current states
      ll du = dp[u], dv = dp[v];
      ll su = sz[u], sv = sz[v];
      // b. Detach v's subtree from u
      dp[u]-=dp[v]+sz[v]; sz[u]-=sz[v];
      // c. Attach u's part under v
      dp[v]+=dp[u]+sz[u]; sz[v]+=sz[u];
      // d. Recurse with v as new root
      dfs2(v, u);
      // e. Rollback for next child
      dp[u] = du; dp[v] = dv;
      sz[u] = su; sz[v] = sv;
   }
}
```

(12.10) Binary Lifting for LCA

```
/**
 * Time Complexity:
 *  - Preprocessing (DFS + Table build)
     : O(N log N)
 *  - Query (LCA): O(log N)
 * Assumes 1-based indexing of nodes.
 */
struct LCA {
int N, logN;
vector<vector<int>> g, parents;
vector<int> depth;
LCA(int n, vector<vector<int>> &v) {
    N = n;
    logN = 32 - __builtin_clz(n);
```

```
    parents.assign(n + 1, vector<int> (
        logN, 0));
    g.assign(n + 1, {});
    depth.assign(n + 1, 0);
    dfs(1, 0);
}
void dfs(int node, int par) {
    depth[node] = depth[par] + 1;
    parents[node][0] = par;
    for (int i = 1; i < logN; ++i) {
        parents[node][i] = parents[
            parents[node][i - 1]][i -
            1];
    }
    for (auto &child : g[node]) {
        if (child != par) {
            dfs(child, node);
        }
    }
}
int getKthPar(int node, int k) {
    for (int i = logN; i >= 0; i--) {
        if ((k >> i) & 1) {
            node = parents[node][i];
        }
    }
    return node;
}
int lca(int a, int b) {
    if (depth[a]<depth[b])swap(a, b);
    int diff = depth[a] - depth[b];
    for (int i = 0; i < logN; ++i) {
        if ((diff >> i) & 1) {
            a = parents[a][i];
        }
    }
    if (a == b) return a;
    for (int i = logN - 1; ~i; --i) {
        if (parents[a][i] != parents[b
            ][i]) {
            a = parents[a][i];
            b = parents[b][i];
        }
    }
    return parents[a][0];
}
};
```

(12.11) HLD (Edge Weight)

include IterativeSegmentTree.cpp

```
/**
 * Time Complexity:
 *  - Preprocessing : O(N)
 *  - Query/Update : O(log N)
 * Use: Tree (Range Max Query)
 * Include Iterative Segment Tree to
     Find Min/Max in a range.
 */
struct HeavyLight {
vector<vector<int>> g;
vector<int> u, v, depth, parents, heavy
    , root, start, weigth;
int timer;
HeavyLight(int n, vector<vector<int>> &
    graph, vector<int> &_u, vector<int>
     &_v, vector<int> &cost, segTree &t
    ) {
    g = graph;
    u = _u; v = _v;
    weigth.resize(n + 1, 0); depth.
        resize(n + 1, 0);
    parents.resize(n + 1, 0); heavy.
        resize(n + 1, 0);
    root.resize(n + 1, 0); start.resize
        (n + 1, 0);
    timer = 0;

    dfs(1, 0);
    decompose(1, 0, 1);

    for (int i = 1; i < n; ++i) {
        int x = u[i];
        int y = v[i];
        if (parents[x] == y)
            weigth[x] = cost[i];
        else
            weigth[y] = cost[i];
    }
    for (int i = 1; i <= n; ++i) {
        t.upd(start[i], weigth[i], n);
    }
}
int dfs(int node, int par) {
    int subTree=1, maxChildSize=0;
    depth[node] = depth[par] + 1;
    parents[node] = par;
    for (auto &child : g[node]) {
        if (child != par) {
            int childSize = dfs(child,
                node);
            subTree += childSize;
            if (childSize >
                maxChildSize) {
                maxChildSize=childSize;
                heavy[node]=child;
            }
        }
    }
    return subTree;
}
void decompose(int node, int par, int
    top) {
    root[node] = top;
    start[node] = timer++;
    if (heavy[node])
        decompose(heavy[node], node,
            top);
    for (auto &child : g[node]) {
        if (child != par && child !=
            heavy[node]) {
            decompose(child, node,
                child);
        }
    }
}
int getMax(int a, int b, int n, segTree
     &t) {
    int ans = 0;
    while (root[a] ^ root[b]) {
        if (depth[root[a]] < depth[root
            [b]])
            swap(a, b);
        ans = max(ans, t.qry(start[root
            [a]], start[a], n));
        a = parents[root[a]];
    }
    if (depth[a] > depth[b]) swap(a, b)
        ;
    if (a != b) {
        // start[a]+1 = (exclude LCA
            for Edge Weight)...
        ans = max(ans, t.qry(start[a] +
             1, start[b], n));
    }
    return ans;
}
void upd(int indx, int val, int n,
    segTree &t) {
    int x = u[indx];
    int y = v[indx];
    if (parents[x] == y)
        indx = start[x];
    else
        indx = start[y];
    t.upd(indx, val, n);
}
};
void Solve() {
    int n; cin >> n;
    vector<int> u(n + 1), v(n + 1),
        cost(n + 1);
    vector<vector<int>> g(n + 1);
    for (int i = 1; i < n; ++i) {
        cin >> u[i] >> v[i] >> cost[i];
        int x = u[i]; int y = v[i];
        g[x].push_back({y});
        g[y].push_back({x});
    }
    segTree t(n);
    HeavyLight hld(n, g, u, v, cost, t)
        ;
    hld.getMax(L, R, n, t);
    hld.upd(indx, val, n, t);
}
```

(12.12) Centroid Decomposition

```
/**
 * Time Complexity:
 *  - Preprocessing (dfs +
     decomposition): O(N log N)
 *  - Each centroid level recurses on
     at most half the subtree.
 * Use: Centroid Decomposition on Tree
     (for divide & conquer on trees)
 */
vector<vector<int>> g;
vector<int> subtree, distRED;
vector<bool> vis;
vector<vector<arr<int, 2>>> ancestors;
void reset(int n) {
    g.assign(n + 1, {});
    subtree.assign(n + 1, 0);
    distRED.assign(n + 1, 2e9); // fill
        with inf
    vis.assign(n + 1, 0);
    ancestors.assign(n + 1, {});
}
void dfs(int node, int par) {
    subtree[node] = 1;
    for (auto &child : g[node]) {
        if (child != par && !vis[child
            ]) {
            dfs(child, node);
            subtree[node] += subtree[
                child];
        }
    }
}
int getCentroid(int node, int par, int
    new_subtree) {
    int centroid = node;
    for (auto &child : g[node]) {
        if (child != par && !vis[child]
            && (subtree[child] * 2 >
            new_subtree)) {
            return getCentroid(child,
                node, new_subtree);
        }
    }
    return centroid;
}
```

```
void getDist(int node, int par, int
    centroid, int curr_dist) {
    ancestors[node].push_back({centroid
        , curr_dist});
    for (auto &child : g[node]) {
        if (child != par && !vis[child
            ]) {
            getDist(child, node,
                centroid, curr_dist +
                1);
        }
    }
}
void buildCentroid(int node, int par) {
    dfs(node, 0);
    int centroid = getCentroid(node,
        par, subtree[node]);
    vis[centroid] = 1;
    for (auto &child : g[centroid]) {
        if (!vis[child]) {
            getDist(child, centroid,
                centroid, 1);
        }
    }
    for (auto &child : g[centroid]) {
        if (!vis[child]) {
            buildCentroid(child,
                centroid);
        }
    }
}
void upd(int node) {
    distRED[node] = 0;
    for (auto &[child, child_dist] :
        ancestors[node]) {
        distRED[child] = min(distRED[
            child], child_dist);
    }
}
int qry(int node) {
    int ans = distRED[node];
    for (auto &[child, curr_dist] :
        ancestors[node]) {
        ans = min(ans, distRED[child] +
             curr_dist);
    }
    return ans;
}
// Usage
reset(n);
buildCentroid(1, 0);
upd(1);   qry(node);
```

Found it

(13) Data Structure

(13.1) Ordered Set (PBDS)

```
#include<ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
template<class T> using oset = tree<T,
    null_type, less<T>, rb_tree_tag,
    tree_order_statistics_node_update>;
void example() {
  // find_by_order(x): return the
      iterator/index of x....
    // order_of_key(x): return the
        count of values less/greater
        than x....
  oset<int> t, t2; t.insert(8);
  auto it = t.insert(10).first;
  assert(it == t.lower_bound(9));
  assert(t.order_of_key(10) == 1);
  assert(*t.find_by_order(0) == 8);
  t.join(t2); // assume t<t2 or t>t2,
      merge t2 into t
    //To Delete (less_equal)
    s.erase(s.find_by_order(s.
        order_of_key(k)));
}
```

(13.2) Fenwick Tree 2D

```
vector<int> bit, v;
int n;
void update(int i2, int j2, int val) {
    for (int i = i2; i <= n; i += (i &
        -i)) {
        for (int j = j2; j <= n; j += (
            j & -j)) {
            bit[i][j] += val;
        }
    }
}
ll qry(int i2, int j2) {
    ll ans = 0;
    for (int i = i2; i > 0; i -= (i & -
        i)) {
        for (int j = j2; j > 0; j -= (j
            & -j))
            ans += bit[i][j];
    }
    return ans;
}
// Usage:
// bit.assign(n + 1, 0);
// update(x, y, 1);
// ll sum = qry(x2, y2) - qry(x2, y -
    1) - qry(x - 1, y2) + qry(x - 1, y
    - 1);
```

(13.3) Sparse Table

```
/**
 * Time: O(NlogN)
 * Supports static range min/max/gcd.
 */
template <class T>
struct RMQ {
vector<vector<T>> sp;
RMQ() {}
RMQ (vector<T> &v) : sp(1, v) {
    int n = sz(v);
    for (int i = 1, p = 1; p * 2 <= n;
        p *= 2, ++i) {
        sp.emplace_back(n-p*2+1);
        for (int j = 0; j < sz(sp[i]);
            ++j) {
            sp[i][j] = min(sp[i - 1][j
                ], sp[i - 1][j + p]);
        }
    }
}
T qry(int L, int R) {
    R += 1; // exclusive
    int len = 31 - __builtin_clz(R - L)
        ;
    return min(sp[len][L], sp[len][R -
        (1 << len)]);
}
};
// Uasage...
RMQ rmq(v);
rmq.qry(L, R); // 0'based;
```

(13.4) Mo's Algorithm

```
/**
 * Time Complexity:
 *  - Build: O(n sqrt (q))
 * Supports offline query.
 */
const int mxN = 2e5+10, mxQ = 1e6+10;
vector<int> v(mxN), cnt(mxN);
int n, q, disCnt = 0;;

struct Node {
int lb, rb, block, ans, indx;
bool operator<(const Node &Q) const {
    if (block == Q.block) {
        return (block & 1 ? rb > Q.rb :
            rb < Q.rb);
    }
    return (block < Q.block);
}
} query[mxQ];

void add(int &x) {
    cnt[x]++;
    if (cnt[x] == 1) disCnt++;
}
void remove(int &x) {
    cnt[x]--;
    if (cnt[x] == 0) disCnt--;
}
void Din() {
    cin >> n >> q;
    for (int i = 0; i < n; ++i) {
        cin >> v[i];
    }
    vector<int> temp(v.begin(), v.begin
        ()+n);
    sort(all(temp));
    temp.erase(unique(all(temp)), temp.
        end());
    for (int i = 0; i < n; ++i) {
        v[i] = lower_bound(all(temp), v
            [i]) - temp.begin()+1;
    }
    int len = (int) sqrtl(n) + 1;
    for (int i = 0; i < q; ++i) {
        cin>>query[i].lb>>query[i].rb;
        query[i].lb--, query[i].rb--;
        query[i].block = query[i].lb/
            len;
        query[i].indx = i;
    }
    sort(query, query + q);
    int L = 0, R = 0;
    for (int i = 0; i < q; ++i) {
        while (R <= query[i].rb)
            add(v[R++]);
        while (R > query[i].rb + 1)
            remove(v[--R]);
        while (L < query[i].lb)
            remove(v[L++]);
        while (L > query[i].lb)
            add(v[--L]);
        int indx = query[i].indx;
        query[indx].ans = disCnt;
    }
    for (int i = 0; i < q; ++i) {
        cout << query[i].ans << '\n';
    }
}
```

(13.5) Segment Tree (Iterative)

```
struct segTree {
int n; vector<int> sg;
// n += 1, for 1'based index
```

```
segTree(int _n) : n(_n+1), sg(4 * n +
    1) {}
void upd(int i, int val, int n) {
    sg[i += n] = val;
    for (i /= 2; i > 0; i /= 2)
        sg[i] = max(sg[2*i],sg[2*i+1]);
}
int qry(int L, int R) {
    int ans = 0;; L += n, R += n+1;
    for (; L < R; L /= 2, R /= 2) {
        if (L&1) ans=max(ans,sg[L++]);
        if (R&1) ans=max(ans,sg[--R]);
    }
    return ans;
}
};
```

(13.6) Lazy Propagation

```
struct Node {
ll sum, maxSubSum, prefix, suffix;
};
struct segTee {
vector<Node> sg; vector<ll> lz;
segTee(int n) : sg(4 * n + 1), lz(4 * n
     + 1, 1e18) {}
void Apply(int s, int e, ll val, int i)
     {
    ll new_val = 1ll * val * (e - s +
        1);
    sg[i].sum = new_val;
    if (val < 0) {
        new_val = 0;
    }
    sg[i].maxSubSum = new_val;
    sg[i].prefix = new_val;
    sg[i].suffix = new_val;
    lz[i] = val;
}
void Push(int s, int mid, int e, int i)
     {
    if (lz[i] != 1e18) {
        Apply(s, mid, lz[i], 2 * i);
        Apply(mid + 1, e, lz[i], 2 * i
            + 1);
    } lz[i] = 1e18;
}
Node merge(Node a, Node b) {
    ll maxSubSum = max({a.maxSubSum, b.
        maxSubSum, a.suffix + b.prefix
        });
    ll sum = a.sum + b.sum;
    ll prefix = max(a.prefix, a.sum + b
        .prefix);
    ll suffix = max(b.suffix, b.sum + a
        .suffix);
    return {sum, maxSubSum, prefix,
        suffix};
}
void upd(int lb, int rb, ll val, int s,
     int e, int i = 1) {
    if (s > rb || e < lb) return;
    if (s >= lb && e <= rb)
        Apply(s, e, val, i);
    else {
        int mid = (s + e) / 2;
        Push(s, mid, e, i);
        upd(lb, rb, val, s, mid, 2 * i)
            ;
        upd(lb, rb, val, mid + 1, e, 2
            * i + 1);
        // sg[i] = min(sg[2 * i], sg[2
            * i + 1]);
        sg[i] = merge(sg[2 * i], sg[2 *
             i + 1]);
    }
}
Node qry(int lb, int rb, int s, int e,
    int i = 1) {
    if (s > rb || e < lb) return {0, 0,
         0, 0};
    if (s >= lb && e <= rb) return sg[i
        ];
    int mid = (s + e) / 2;
    Push(s, mid, e, i);
    return merge(qry(lb, rb, s, mid, 2
        * i) , qry(lb, rb, mid + 1, e,
        2 * i + 1));
}
};
// usage...
    segTee t(n);
    t.upd(lb, rb, val, 1, n);
    t.qry(1, n, 1, n).maxSubSum
}
```

(13.7) Persistent Segment Tree

```
vector<int> v, root;
struct Node {
    ll sum; int lc, rc;
    Node () : sum(0), lc(0), rc(0) {}
};
class persistentSegTree {
    vector<Node> sg;  int RT;
public: // Alert 40*n for PST
persistentSegTree (int _n) : sg(40*_n
    +1), RT(1) {}
int build (int s, int e) {
    int cur = RT++; // root
    if (s == e) return cur;
    int mid = (s + e) / 2 ;
    sg[cur].lc = build(s, mid);
    sg[cur].rc = build(mid+1, e);
    sg[cur].sum = sg[sg[cur].lc].sum ^
        sg[sg[cur].rc].sum;
    return cur;
}
int upd (int prev, int indx, ll val,
    int s, int e) {
    int cur = RT++;
    sg[cur] = sg[prev];
    if (s == e) {
        sg[cur].sum ^= val;
        return cur;
    }
    int mid = (s + e) / 2;
    if (indx <= mid) {
        sg[cur].lc = upd(sg[prev].lc,
            indx, val, s, mid);
    }
    else {
        sg[cur].rc = upd(sg[prev].rc,
            indx, val, mid+1, e);
    }
    sg[cur].sum = sg[sg[cur].lc].sum ^
        sg[sg[cur].rc].sum;
    return cur;
}
int qry (int prev, int cur, int s, int
    e) {
    if ((sg[prev].sum ^ sg[cur].sum) ==
         0) return 0;
    if (s == e) return s;
    int mid = (s + e) / 2;
    ll leftXor = sg[sg[prev].lc].sum ^
        sg[sg[cur].lc].sum;
    if (leftXor != 0) {
        return qry(sg[prev].lc, sg[cur
            ].lc, s, mid);
    }
    else {
        return qry(sg[prev].rc, sg[cur
            ].rc, mid+1, e);
    }
}
};
// usage:
persistentSegTree t(n);
root[0] = t.build(1, n);
for (int i = 1; i <= n; ++i) {
    int id = lower_bound(all(sorted_v),
         v[i])-begin(sorted_v);
    root[i] = t.upd(root[i-1], id, hash
        [id], 1, n);
}
int indx = t.qry(root[L], root[R], 1, n
    );
```

(14) Dynamic Programming

(14.1) 0/1 Knapsack

Time: $O(n \cdot W)$, **Space:** $O(W)$

```
ll knap01(int n, int W,
          vector<int>& wt, vector<ll>&
              val){
  vector<ll> dp(W + 1, 0);
  for (int i = 0; i < n; i++)
    for (int w = W; w >= wt[i]; w--)
      dp[w] = max(dp[w],
                  dp[w - wt[i]] + val[i
                      ]);
  return dp[W];
}
```

(14.2) Coin Change (# Ways)

Time: $O(n \cdot W)$, **Space:** $O(W)$

```
// Outer coins = Combinations (unorder)
// Outer W     = Permutations (ordered)
ll coinWays(vector<int>& coins, int W)
    {
  vector<ll> dp(W + 1, 0);
  dp[0] = 1;
  for (int c : coins)
    for (int w = c; w <= W; w++)
      dp[w] = (dp[w] + dp[w - c]) % mod
          ;
  return dp[W];
}
```

(14.3) LIS

Time: $O(n \log n)$, **Space:** $O(n)$

```
// lower_bound: strictly incr (<)
// upper_bound: non-decreasing (<=)
int LIS(vector<ll>& a) {
  vector<ll> v;
  for (ll x : a) {
    auto it = lower_bound(
      v.begin(), v.end(), x);
    if (it == v.end()) v.push_back(x);
    else *it = x;
  }
  return v.size();
}
```

(14.4) LCS + Reconstruction

Time/Space: $O(n \cdot m)$

```
string lcs(string &a, string &b) {
  int n = a.size(), m = b.size();
  vector<vector<int>> dp(n + 1,
```

```
    vector<int>(m + 1, 0));
  for (int i = 1; i <= n; i++)
    for (int j = 1; j <= m; j++)
      if (a[i - 1] == b[j - 1])
        dp[i][j] = dp[i-1][j-1] + 1;
      else
        dp[i][j] = max(dp[i - 1][j],
                       dp[i][j - 1]);
  string res = "";
  int i = n, j = m;
  while (i > 0 && j > 0) {
    if (a[i - 1] == b[j - 1]) {
      res += a[i - 1]; i--; j--;
    } else if (dp[i-1][j] >= dp[i][j
        -1])
      i--;
    else j--;
  }
  reverse(res.begin(), res.end());
  return res;
}
```

(14.5) Bounded Knapsack (Pref DP)

Ways to form sum x; item i has value w_i, count c_i. (Set $w_i = 1$ for simple bounded sum). **Time:** $O(nx)$

```
ll boundedWays(int n, int x,
    vector<int>& w, vector<int>& c) {
  vector<ll> dp(x + 1, 0), pr(x + 1, 0)
      ;
  dp[0] = 1;
  for (int i = 0; i < n; i++) {
    for (int k = 0; k <= x; k++)
      pr[k] = (dp[k] + (k >= w[i] ?
                pr[k - w[i]] : 0)) % mod
                  ;
    for (int k = 0; k <= x; k++) {
      int L = k - (c[i] + 1) * w[i];
      ll sub = (L >= 0 ? pr[L] : 0);
      dp[k] = (pr[k] - sub + mod) % mod
          ;
    }
  }
  return dp[x];
}
```

(14.6) Bitmask, Submask & SOS DP

```
// 1. Assignment DP: O(n*2^n), Mem O(2^
    n)
const ll INF = 1e18;
vector<ll> dp(1 << n, -INF);
dp[0] = 0;
for (int m = 0; m < (1 << n); m++) {
  if (dp[m] == -INF) continue;
  int r = __builtin_popcount(m);
  for (int c = 0; c < n; c++)
    if (!(m & (1 << c))) {
      int nm = m | (1 << c);
      dp[nm] = max(dp[nm],
                   dp[m] + v[r][c]);
    }
}
// 2. Submasks of all masks: O(3^n)
// Excludes s=0; handle s=0 after loop
for (int m = 0; m < (1 << n); m++)
  for (int s = m; s > 0; s = (s-1) & m)
    { /* use s */ }

// 3. SOS DP: O(n * 2^n)
for (int i = 0; i < n; i++)
  for (int m = 0; m < (1 << n); m++)
    if (m & (1 << i))
      F[m] += F[m ^ (1 << i)];
```

(14.7) Digit DP

Time: $O(\text{len} \cdot \text{state} \cdot 10)$

```
string S;
// 20 digits, max digit sum 9*20 = 180
ll dp[20][200][2]; // [pos][sum][
    started]
ll call(int p, int sum, bool ti, bool
    st){
  if (p == (int)S.size())
    return sum == target; // 0 has sum
        0
  ll &res = dp[p][sum][st];
  if (!ti && res != -1) return res;
  ll ans = 0;
  int lim = ti ? (S[p] - '0') : 9;
  for (int d = 0; d <= lim; d++) {
    bool n_ti = ti && (d == lim);
    bool n_st = st || (d > 0);
    int n_sum = n_st ? (sum + d) : 0;
    ans += call(p+1, n_sum, n_ti, n_st)
        ;
  }
  if (!ti) res = ans;
  return ans;
}
ll solve(ll x) {
  if (x < 0) return 0;
  S = to_string(x);
  memset(dp, -1, sizeof(dp));
  return call(0, 0, 1, 0);
}
```

(14.8) Divide & Conquer DP

0-indexed: $dp_{cur}[i] = \min_{s-1 \le k \le i}\{dp_{pre}[k-1] + C(k,i)\}$ at layer s.
Needs $opt[i] \le opt[i+1]$. **Time:** $O(KN \log N)$

```
vector<ll> dp_pre(n), dp_cur(n);
void calc(int l, int r, int oL, int oR)
    {
  if (l > r) return;
  int mid = (l + r) >> 1;
  pair<ll, int> best = {INF, -1};
  int lim = min(mid, oR);
  for (int k = max(1, oL); k <= lim; k
      ++)
    best = min(best,
      {dp_pre[k - 1] + C(k, mid), k});
  dp_cur[mid] = best.first;
  int opt = best.second;
  calc(l, mid - 1, oL, opt);
  calc(mid + 1, r, opt, oR);
}
// Layer s=1: dp_pre[i] = C(0, i)
// For s=2..K: fill dp_cur with INF;
//   calc(s-1, n-1, s-1, n-1);
//   dp_pre = dp_cur;
```

(14.9) Knuth's Optimization

1-indexed: $dp[l][r] = \min_{l \le k < r}\{dp[l][k] + dp[k+1][r]\} + C(l,r)$.
Needs $opt[l][r-1] \le opt[l][r] \le opt[l+1][r]$. **Time:** $O(N^2)$

```
for (int len = 1; len <= n; len++) {
  for (int l = 1; l+len-1 <= n; l++) {
    int r = l + len - 1;
    if (len == 1) {
      dp[l][r] = 0; opt[l][r] = l;
      continue;
    }
    dp[l][r] = INF;
    int oL = max(l, opt[l][r - 1]);
    int oR = min(r - 1, opt[l + 1][r]);
    for (int k = oL; k <= oR; k++) {
      ll v = dp[l][k]+dp[k+1][r]+C(l,r)
          ;
      if (v < dp[l][r]) {
        dp[l][r] = v; opt[l][r] = k;
      }
    }
  }
}
```

(15) String Algorithm

(15.1) KMP String Matching

```
vector<int> LPS(string &s, int n) {
    vector<int> ans(n, 0);
    for (int i = 1; i < n; ++i) {
        int g = ans[i - 1];
        while (g && s[i] != s[g]) g =
            ans[g - 1];
        ans[i] = g + (s[i] == s[g]);
    }
    return ans;
}
void KMP() {
    string s, pat;
    cin >> s >> pat;
    s = pat + "$" + s;
    int n = sz(s), m = sz(pat);
    vector<int> ans = LCP(s, n);
    for (int i = 0; i < sz(s); ++i) {
        if (ans[i] == m)
            cout << i - 2 * m << '\n';
    }
}
```

(15.2) Z-Function

```
/**
 * Time: O(N)
 *  z[i] computes the length of the
     longest common prefix of s[i:] and
      s.
 */
vi Z(const string &s, int n) {
    vi v(n); // v[0]=n, full str match
    int L = -1, R = -1;
    for(i, 1, n) {
        v[i] = i >= R ? 0 : min(R - i,
            v[i-L]);
        while (i + v[i] < n && s[i + v[
            i]] == s[v[i]]) {v[i] +=
            1;}
        if (i + v[i] > R) {
            L = i, R = i + v[i];
        }
    }
    return v;
}
//Use: auto ans = Z(s, n);
```

(15.3) Manacher

```
/**
 * Time: O(N)
```

```
 *  For each position, p[0][i] = half
    length of longest even palindrome
    around pos i, p[1][i] = longest
    odd (rounded down).
 */
arr<vi, 2> manacher(string &s, int n) {
    arr<vi, 2> p = {vi(n + 1), vi(n)};
    for(z, 0, 2) {
        for (int i = 0, lb = 0, rb = 0;
             i < n; ++i) {
            int t = rb - i + !z;
            if (i < rb) p[z][i] = min(t
                , p[z][lb + t]);
            int L = i - p[z][i], R = i
                + p[z][i] - !z;
            while (L >= 1 && R + 1 < n
                && s[L - 1] == s[R +
                1]) {
                p[z][i]++, L--, R++;
            }
            if (R > rb) lb = L, rb = R;
        }
    }
    return p;
}
void Din() {
    string s; cin >> s; int n = sz(s);
    auto [even, odd] = manacher(s, n);
    int len = 1, pos = 0;
    for (int i = 0; i < n; ++i) {
        if (even[i]) {
            int curr_len = 2 * even[i];
            if (curr_len > len) {
                len = curr_len;
                pos = i - even[i];
            }
        }
        if (odd[i]) {
            int curr_len = 2 * odd[i] +
                1;
            if (curr_len > len) {
                len = curr_len;
                pos = i - odd[i];
            }
        }
    }
    cout << s.substr(pos, len) << "\n";
}
```

(15.4) Min-Rotation

```
// * Time: O(N)
// * Finds the lexicographically
    smallest rotation of a string.
int minRotation(string s, int n) {
    int a = 0; s += s;
    for(b, 0, n) for(k, 0, n) {
        if (a + k == b || s[a + k] < s[
            b + k]) {b += max(0, k - 1)
            ; break;}
        if (s[a + k] > s[b + k]) { a =
            b; break; }
    }
    return a;
}
// Usage: (n = orginal size)
// int pos = minRotation(s, n);
// s += s, s.substr(pos, n)
```

(15.5) String Hashing

```
typedef uint64_t ull;
struct H {
    ull x; H(ull x=0) : x(x) {}
    H operator+(H o) const { return x +
        o.x + (x + o.x < x); }
    H operator-(H o) const {return *
        this+~o.x;}
    H operator*(H o) const { auto m = (
        __uint128_t)x * o.x;
        return H((ull)m) + (ull)(m >>
            64); }
    ull get() const { return x + !~x; }
    bool operator==(H o) const { return
        get() == o.get(); }
    bool operator<(H o) const { return
        get() < o.get(); }
};
static const H prime = (ll)1e11 + 3; //
    (order ~ 3e9; random also ok)
struct HashInterval {
vector<H> hsh, pw;
HashInterval(string& s) : hsh(sz(s)+1),
    pw(hsh) {
    pw[0] = 1;
    for (int i = 0; i < sz(s); ++i) {
        hsh[i+1]=hsh[i]*prime+s[i];
        pw[i+1] = pw[i]*prime;
    }
}
H qry(int a, int b) {// (exc, inc)
    return hsh[b]-hsh[a]*pw[b-a];
}
};
vector<H> getHashes(string& s, int
    length) {
    if (sz(s) < length) return {};
    H h = 0, pw = 1;
    for (int i = 0; i < length; ++i) {
        h=h*prime+s[i], pw = pw*prime;
    }
    vector<H> ans = {h};
    for (int i = length; i < sz(s); ++i
        ) {
        ans.push_back(h = h * prime + s
            [i] - pw * s[i - length]);
    }
    return ans;
}
H hashString(string& s) {
    H h{}; for (char c : s) h = h*prime
        +c; return h;
}
// Usage:
// hashInterval prefix(s);
// reverse(all(s));
// hashInterval suffix(s);
// prefix.qry(L, R)
// suffix.qry(N-R, N-L)
// Prepend: hsh = H(ch) * pw[len] + hsh
// Append:  hsh = hsh * prime + ch
// PopFront: hsh = hsh-H(ch)*pw[len-1]
// PopBack: (hsh - ch) * invPrime
// H invPrime = binExp(prime, (1ULL
    <<63)-1);
```

(15.6) Dynamic String Hashing

```
// #include String Hash Struct
struct hashInterval {
struct Node {
    H hsh = 0; int len = 0;
};
int n; vector<H> pw; vector<Node> sg;
hashInterval (string &s) {
    n = sz(s); sg.resize(4*n+1);
    pw.resize(n+1); pw[0] = 1;
    for (int i = 1; i <= n; ++i)
        pw[i] = pw[i-1] * prime;
    for (int i = 1; i <= n; ++i)
        upd(i, s[i-1], 1, n);
}
Node merge (const Node &a, const Node &
    b) {
    Node temp;
    temp.hsh = a.hsh * pw[b.len] + b.
        hsh;
    temp.len = a.len + b.len;
    return temp;
}
void upd (int idx, char val, int s, int
    e, int i = 1) {
    if (s == e) {
        sg[i] = {val, 1}; return;
    }
    int mid = (s + e) / 2;
    if (idx <= mid)
        upd(idx, val, s, mid, 2*i);
    else
        upd(idx, val, mid+1, e, 2*i+1);
    sg[i] = merge(sg[2*i], sg[2*i+1]);
}
Node qry (int lb, int rb, int s, int e,
    int i = 1) {
    if (s>rb || e<lb) return{0, 0};
    if (s>=lb && e<=rb) return sg[i];
    int mid = (s + e) / 2;
    return merge(qry(lb, rb, s, mid, 2*
        i), qry(lb, rb, mid+1, e, 2*i
        +1));
}
};
// Usage:
// hashInterval prefix(s);
// reverse(all(s));
// hashInterval suffix(s);
// prefix.upd(idx, ch, 1, n);
// suffix.upd(n-idx+1, ch, 1, n);
// H hsh1 = prefix.qry(lb, rb, 1, n).
    hsh;
// H hsh2 = suffix.qry(n-rb+1, n-lb+1,
    1, n).hsh;
```

(15.7) Trie

```
/*
Features:
- Count exact occurrences of a word (
    countEqual)
- Count words starting with a prefix (
    countPrefix)
- Check if a word or prefix exists (
    countEqual > 0 / countPrefix > 0)
Time: O(|word|) per operation.
*/
struct Trie {
static const int alpha = 26;
struct Node {
    int nxt[alpha], end=0, prefix=0;
};
vector<Node> t;
Trie() {
    t.emplace_back();
    memset(t[0].nxt, -1, sizeof(t[0].
        nxt));
}
int newNode() {
    t.emplace_back();
    memset(t.back().nxt, -1, sizeof(t.
        back().nxt));
    return (int)t.size() - 1;
}
void insert(const string &s) {
    int cur = 0;
```

```
    for (char c : s) {
        int x = c - 'a';
        if (t[cur].nxt[x] == -1) {
            int indx = newNode();
            t[cur].nxt[x] = indx;
        }
        cur = t[cur].nxt[x];
        t[cur].prefix++;
    }
    t[cur].end++;
}
void erase(const string &s) {
    int cur = 0;
    vector<int> path = {0};
    for (char c : s) {
        int x = c - 'a';
        if (t[cur].nxt[x] == -1) return
            ;
        cur = t[cur].nxt[x];
        path.push_back(cur);
    }
    if (t[cur].end == 0) return;
    t[cur].end--;
    for (int i = 1; i < (int)path.size
        (); i++) t[path[i]].prefix--;
}
int countEqual(const string &s) {
    int cur = 0;
    for (char c : s) {
        int x = c - 'a';
        if (t[cur].nxt[x] == -1) return
             0;
        cur = t[cur].nxt[x];
    }
    return t[cur].end;
}
int countPrefix(const string &s) {
    int cur = 0;
    for (char c : s) {
        int x = c - 'a';
        if (t[cur].nxt[x] == -1) return
             0;
        cur = t[cur].nxt[x];
    }
    return t[cur].prefix;
}
};
// Usage:
// Trie t;
// t.startsWith(pat), t.countPrefix(pat
    )
```

(15.8) AhoCorasick

```
/*
Features:
- Match multiple patterns in a text
- Count occurrences of each pattern (
    getCnt)
- Find starting indices of all matches
    (findAll)
- Find longest match ending at each
    position (find)
- Count total matches of all patterns
    combined (nmatches)
Time: O(|text| + sum(|pat|))
*/
struct AhoCorasick {
enum {alpha = 26, first = 'a'}; //
    change this!
struct Node {
    // (nmatches is optional)
    int back, next[alpha], start = -1,
        end = -1, nmatches = 0;
    Node(int v) { memset(next, v,
        sizeof(next)); }
};
vector<Node> N;
vector<int> backp, bfsOrd, patNode;
void insert(string& s, int j) {
    assert(!s.empty());
    int n = 0;
    for (char c : s) {
        int& m = N[n].next[c - first];
        if (m == -1) { n = m = sz(N); N
            .emplace_back(-1); }
        else n = m;
    }
    if (N[n].end == -1) N[n].start = j;
    backp.push_back(N[n].end);
    N[n].end = j;
    N[n].nmatches++;
    patNode.push_back(n);
}
AhoCorasick(vector<string>& pat) : N(1,
     -1) {
    for (int i = 0; i < sz(pat); ++i)
        insert(pat[i], i);
    N[0].back = sz(N);
    N.emplace_back(0);
    queue<int> q;
    for (q.push(0);!q.empty();q.pop()){
        int n = q.front(), prev = N[n].
            back;
        bfsOrd.push_back(n);
        for (int i = 0; i<alpha; ++i) {
            int &ed = N[n].next[i], y =
                 N[prev].next[i];
            if (ed == -1) ed = y;
            else {
                N[ed].back = y;
                N[ed].end == -1 ? N[ed
                    ].end : backp[N[ed
                    ].start])
                    = N[y].end;
                N[ed].nmatches += N[y].
                    nmatches;
                q.push(ed);
            }
        }
    }
}
vector<int> find(string& s) {
    int n = 0;
    vector<int> ans; // ll count = 0;
    for (char c : s) {
        n = N[n].next[c - first];
        ans.push_back(N[n].end);
        // count += N[n].nmatches;
    }
    return ans;
}
vector<vector<int>> findAll(vector<
    string>& pat, string& s) {
    vector<int> r = find(s);
    vector<vector<int>> ans(sz(s));
    for (int i = 0; i < sz(s); ++i) {
        int ind = r[i];
        while (ind != -1) {
            ans[i - sz(pat[ind]) + 1].
                push_back(ind);
            ind = backp[ind];
        }
    }
    return ans;
}
vector<int>getCnt(int m, string& s) {
    vector<int> nodeCnt(sz(N), 0);
    int n = 0;
    for (char c : s) {
        n = N[n].next[c - first];
        nodeCnt[n]++;
    }
    for (int i = sz(bfsOrd) - 1; i > 0;
         --i) {
        int u = bfsOrd[i];
        nodeCnt[N[u].back] += nodeCnt[u
            ];
    }
    vector<int> ans(m, 0);
    for (int i = 0; i < m; ++i) {
        ans[i] = nodeCnt[patNode[i]];
    }
    return ans;
}
};
// Usage: pat = list of patterns
// AhoCorasick aho(pat);
// vector<int> cnt = aho.getCnt(sz(pat)
    , text);
// auto ans = aho.findAll(pat, text);
```

(15.9) Suffix Array

```
// Include SPARSE TABLE if query need
// Pattern matching/Count: Binary
    Search SA
// Longest repeated substr: max(LCP)
// Distinct substr: n*(n+1)/2-sum(LCP)
// LCS(2 strings): SA on s+'#'+t, scan
    LCP & check indx come from s and t
// Distinct substr of length k: ans +=
    max(0, k - LCP[i])
// LCP between any two suffixes: use
    RMQ on LCP
struct suffixArray {// 0 based
string s; int n; vi idx, lcp, rank;
RMQ <int> rmq;
suffixArray(string _s, int lim = 256) {
    s = _s; s.pb(0); n = sz(s);
    int k = 0, a, b; rank = vi(all(s));
    vi temp(n), cnt(max(n, lim));
    idx = lcp = temp; iota(all(idx),0);
    for (int j = 0, p = 0; p < n; j =
        max(1, j * 2), lim = p) {
        p = j; iota(all(temp), n - j);
        for(i, 0, n) {if (idx[i] >= j)
            {temp[p++]=idx[i]-j;}}
        fill(all(cnt), 0);
        for(i,0,n)cnt[rank[i]] += 1;
        for(i,1,lim)cnt[i]+=cnt[i-1];
        for (int i = n; i--;) {
            idx[--cnt[rank[temp[i]]]] =
                temp[i];
        }
        swap(rank, temp); p = 1; rank[
            idx[0]] = 0;
        for(i, 1, n) {
            a = idx[i - 1]; b = idx[i];
            if (temp[a] == temp[b] &&
                temp[a + j] == temp[b +
                j]) {
                rank[b] = p - 1;
            }
            else {rank[b] = p++;}
        }
    }
    // Kasai's Algorithm
    for (int i = 0, j = 0; i < n - 1;
        lcp[rank[i++]] = k) {
        for (k && k--, j = idx[rank[i]
            - 1]; i + k < n && j + k <
            n && s[i + k] == s[j + k];
```

```
            k++);
    }
    temp.clear();
    for(i, 1, n) {temp.pb(lcp[i]);}
    rmq = RMQ<int>(temp);
}
int getLCP(int i) {return lcp[i];}
int getLCP(int i, int j) {
    if (i == j) return n - i - 1;
    i = rank[i], j = rank[j];
    if (i > j) swap(i, j);
    j -= 1; //last lcp excluded..
    return rmq.qry(i, j);
}
arr<int, 2> getLCS(int na) {//in a+b
    int best = 0, pos = -1;
    for (int i = 1; i < n; ++i) {
        bool ok = idx[i-1] < na;
        bool ok2 = idx[i] < na;
        if (lcp[i]>best && ok != ok2) {
            best=lcp[i]; pos=idx[i];
        }
    }
    if (best == 0) return { -1, -1};
    return {pos, best};
}
int getCnt(string &pat) {
    int m = sz(pat), L = 0, R = n;
    for (int k = 0; k < m; ++k) {
        int lb = L, rb = R;
        char ch = pat[k];
        while (lb < rb) {
            int mid = lb + (rb-lb)/2;
            int pos = idx[mid] + k;
            if (pos<n && s[pos] >= ch)
                rb = mid;
            else lb = mid + 1;
        }
        L = lb; lb = L, rb = R;
        while (lb < rb) {
            int mid = lb + (rb-lb)/2;
            int pos = idx[mid] + k;
            if (pos < n && s[pos] > ch)
                rb = mid;
            else lb = mid + 1;
        }
        R = lb;
    }
    return R - L;
}
};
//Usage: [suffixArray t(s), t2(s+"$"+t)
    ]
// auto [pos, len] = t.getLCS(sz(a))
// t.getLCP(a, b), t.getCnt(pat)
```

(16) Game Theory

(16.1) Basic Concepts

- **Impartial Game:** Legal moves depend only on the state, not on who is playing.
- **Sequential:** Players take alternate turns.
- **Winning State:** If it can give opponent a losing state.
- **Losing State:** If from this only winning states can be given to opponent.
- **Strategy:** Move to a losing state for opponent.
- **Example:**
  Rules: Remove 1, 2, or 3 stones. Last move wins.

State	0	1	2	3	4	5	6	7
Type	L	W	W	W	L	W	W	W

Pattern: Losing states every 4 steps (0,4,8,...)

(16.2) Standard Nim Game

Rules: n piles, remove any number from one pile. Last move wins.

Strategy:

- Compute XOR sum of all piles.
- If XOR $\neq 0$, first player wins.
- Else, first player loses.

Example:

- Piles = {5, 7, 13, 9}
- XOR $= 5 \oplus 7 \oplus 13 \oplus 9 = 6 \Rightarrow$ Win
- Make a move to make the XOR 0, which creates a losing state for the opponent (e.g., $13 \to 11$).

(16.3) Misère Nim

Rules: Last move loses.

Strategy:

- All piles 1: Even count $\Rightarrow$ Win, Odd $\Rightarrow$ Lose.
- Otherwise: Standard Nim strategy.

(16.4) Bogus Nim

Rules: Players have pockets of stones they can use. Last move wins.

Strategy: Pockets are irrelevant. Solve using normal XOR.

(16.5) Staircase Nim

Rules: Coins can be moved from stair i to $i-1$. Coins on stair 1 are removed.

Strategy: Only odd-numbered stairs matter. Treat them as Nim piles.

(16.6) Sprague-Grundy Theorem

Theory: Every impartial game is equivalent to a Nim game using Grundy numbers.

Grundy Number: Minimum Excludant (MEX) of all reachable states' Grundy numbers.

Rule:

- Compute Grundy numbers for all piles.
- XOR them.
- If result $\neq 0$, first player wins.

Example (remove up to half):

- $g(0) = 0, g(1) = \text{MEX}\{0\} = 1, g(2) = \text{MEX}\{1\} = 0$
- Grundy{5,2,1,4} = {0,0,1,2} $\Rightarrow$ XOR $= 3 \Rightarrow$ Win

(16.7) Pile Splitting Game

Rules: Split a pile into two unequal parts.

Strategy: Grundy number of pile n is:

$$g(n) = \text{MEX}\{g(i) \oplus g(n-i)\}, \quad 1 \le i < \frac{n}{2}$$

Example:

$$g(5) = \text{MEX}\{g(1) \oplus g(4),\ g(2) \oplus g(3)\}$$

(16.8) Grundy + XOR (Multiple Games)

Theorem: Total Grundy number of a game = XOR of Grundy numbers of subgames.

Win: If XOR $\neq 0$, first player wins.

(16.9) Green Hackenbush (Graph Game)

Concept: Tree/graph game where players remove edges.

Strategy:

$$g(\text{node}) = (g(\text{child}_1) \oplus g(\text{child}_2) \oplus \cdots) + 1$$

(Depends on variant)

(16.10) Beatty's Theorem and Wythoff's Game

Beatty Sequence: For an irrational number $r > 1$, the Beatty sequence B_r is defined as:

$$B_r = \{\lfloor nr \rfloor \mid n = 1, 2, 3, \ldots\}$$

Beatty's Theorem: If $r, s > 1$ are irrational numbers such that

$$\frac{1}{r} + \frac{1}{s} = 1,$$

then the Beatty sequences B_r and B_s partition the positive integers without overlap, i.e., every positive integer appears exactly in one of these sequences.

Wythoff's Game: Two piles. Remove any number from one pile or the same number from both piles. Losing positions follow Beatty sequences.

Losing Positions: The losing states are given by pairs:

$$(A_n, B_n) = \left(\lfloor n\phi \rfloor, \lfloor n\phi^2 \rfloor\right),$$

where $\phi = \frac{1+\sqrt{5}}{2} \approx 1.618$ (the golden ratio).

Example: For $n = 1, 2, 3$:

$$(A_1, B_1) = (\lfloor 1 \times 1.618 \rfloor, \lfloor 1 \times 2.618 \rfloor) = (1, 2),$$
$$(A_2, B_2) = (\lfloor 2 \times 1.618 \rfloor, \lfloor 2 \times 2.618 \rfloor) = (3, 5),$$
$$(A_3, B_3) = (\lfloor 3 \times 1.618 \rfloor, \lfloor 3 \times 2.618 \rfloor) = (4, 7).$$

These pairs represent losing positions where the player to move will lose with optimal play.

(16.11) Other Useful Games

- **Take-Away Game:** Remove 1 to k stones. Last move wins. Losing positions repeat every $(k+1)$.
- **Kayles Game:** Remove 1 or 2 adjacent pins. Uses Sprague-Grundy theorem.

(16.12) Techniques Summary

- **Brute Force:** Try small states to find patterns.
- **Use MEX:** When computing Grundy numbers for a single pile.
- **Use XOR:** When combining multiple independent games.
- **Use Large constraint:** Calculate for little constraint and then find pattern.
- **For pattern:** Calculate by mod for large constraint.

```
if(x >= minMatchInd){
    set<int> patIndex = {0, 12, 16,
        30}; //midMatchInd = 0;
    int mod = HighMatchInd -
        minMatchInd;
    int newInd = (x - minMatchInd) %
        mod;
    if(patIndex.find(newInd)==patInd.
        end()){
        //Win
    }
}
```

(17) Test & Debug

(17.1) Generate Random

gen.cpp

```
#define ll long long
double getDouble(double a, double b) {
    double x = a + static_cast<double>
        (rand()) / RAND_MAX * (b-a);
    return x;
}
mt19937_64 rng(chrono::steady_clock::
    now().time_since_epoch().count());
ll getRand(ll a, ll b) {
    uniform_int_distribution<ll> dis(a,
        b);
    return dis(rng);
}
int main(int argc, char* argv[]) {
    if (argc > 1) srand(atoi(argv[1]));
    int t = 1;
    int n = getRand(1, 5);
    // cout << t << n << '\n';
    set<ll> used;
    for (int i = 0; i < n; ++i) {
        ll x;
        do { x = getRand(1, n);
        } while (used.count(x));
        cout << x << " " << x << '\n';
    }
    return 0;
}
```

(17.2) Tree generator

genTree.cpp

```
// include getRand() function
int main(int argc, char* argv[]) {
    if (argc > 1) srand(atoi(argv[1]));
    int n = getRand(2, 10000);
    // printf("%d\n", n);
    cout << n << '\n';
    vector<pair<int, int>> edges;
    for (int i = 2; i <= n; ++i) {
    edges.emplace_back(getRand(1, i -
        1), i);
    }
    vector<int> perm(n + 1);
    for (int i = 1; i <= n; ++i) {
        perm[i] = i;
    }
    random_device rd;
    mt19937 g(rd());
    shuffle(perm.begin() + 1, perm.end
        (), g);
    shuffle(edges.begin(), edges.end(),
         g);

    for (auto edge : edges) {
        int a = edge.first, b = edge.
            second;
        // if (rand() % 2) {
        //     // random order of two
            vertices
        //     swap(a, b);
        // }
        cout << perm[a] << " " << perm[
            b] << '\n';
        }
    }
```

(17.3) Stress Testing Script

compare.sh

```
for((i = 1; ; ++i));
do
    echo "Test: $i"
    ./gen $i > input.txt
    diff -w <(./g < input.txt) <(./e <
        input.txt) || break
done
```

(17.4) Debug

debug.h

```
void __p(bool x) { cerr<<(x?"1":"0"); }
void __p(char x) { cerr<<x; }
void __p(int x) { cerr<<x; }
void __p(long long x) { cerr<<x; }
void __p(double x) { cerr<<x; }
void __p(const string& x) { cerr<<'"'<<
    x<<'"'; }
void __p(const char *x) { cerr<<'"'<<x
    <<'"'; }
// forward declare before pair so pair<
    A, vector<B>> can find it
template<class T> void __p(const T& x);
template<class A,class B>
void __p(const pair<A,B>& x){ cerr<<"("
    ; __p(x.first); cerr<<","; __p(x.
    second); cerr<<")"; }
template<class T>
void __p(const T& x){
    cerr<<"{"; bool f=0;
    for(auto i:x){ if(f)cerr<<","; __p(
        i); f=1; }
    cerr<<"}";
}
void __db(const char*){ cerr<<"\n"; }
template<class T,class...R>
void __db(const char* n, T v, R...r){
    while(*n==' ')n++;
    const char* c=strchr(n,',');
    cerr<<string(n,c?c-n:strlen(n))<<"
        = "; __p(v); cerr<<" | ";
    __db(c?c+1:"",r...);
}
#define dbg(...) cerr << "[" <<
    __LINE__ << "] ",__db(#__VA_ARGS__,
    __VA_ARGS__)
```

(17.5) Python Basic syntax

```
import sys, os, math, heapq, bisect
from collections import deque,
    defaultdict, Counter
from decimal import Decimal, getcontext
from functools import cmp_to_key
input = lambda: sys.stdin.readline().
    rstrip()
dbg = lambda *a: os.getenv("LOCAL") and
     print("DBG:", *a, file=sys.stderr)
sys.setrecursionlimit(200000)
sys.set_int_max_str_digits(1000000) #
    For >4300 digits
getcontext().prec = 50 # Decimal("1.2")
     / Decimal("7")
def solve():
    n = int(input())
    a, b = map(int, input().split())
    v = list(map(int, input().split()))
    # Math & BigInt
    g, l = math.gcd(a, b), math.lcm(a,
        b)
    sq = math.isqrt(n)
        # Exact floor(sqrt(n))
    ncr, npr = math.comb(n, 2), math.
        perm(n, 2)
    f = math.factorial(20)
    mpow = pow(a, b, 10**9 + 7)
        # (a^b) % m
    minv = pow(a, -1, 10**9 + 7)
        # Mod inv (if gcd=1)
    b_str, val = bin(n)[2:], int("1011"
        , 2)
    # Lists & Sorting
    mn, mx, tot = min(v), max(v), sum(v
        )
    v.sort(key=lambda x: (abs(x), -x))
        # By |x| asc, then x desc
    v.sort(key=cmp_to_key(lambda x, y:
        -1 if x < y else 1))
    grid = [[0] * 5 for _ in range(n)]
        # 2D: n rows, 5 cols
    v.pop(); v.pop(2); v.remove(a)
        # Del last, idx 2, val a
    # Binary Search
    lb = bisect.bisect_left(v, a)
        # 1st idx >= a
    ub = bisect.bisect_right(v, a)
        # 1st idx > a
    # Data Structures
    freq = Counter(v)
        # Or defaultdict(int)
    st = set(v); st.add(10); st.discard
        (5)
    dq = deque(v); dq.append(1); dq.
        appendleft(0)
    dq.pop(); dq.popleft()
    pq = []; heapq.heappush(pq, 10)
        # Min-heap (-x for Max)
    top = heapq.heappop(pq)
    # Output
    dbg(n, v, freq)
        # Auto-disabled on Judge
```

```
    print(*v)
        # Space-separated
    print(f"{mn:.6f}")
def main():
    t = 1
    t = int(input()) # Comment out for
        single testcase
    for tc in range(1, t + 1):
        # print(f"Case {tc}: ", end="")
        solve()
if __name__ == "__main__":
    main()
```